

Full length article

IDORacle: Template-guided data-access mediation for object-level authorization in database-backed applications

Yuewantong Song^{a,1}, Guanhang Shi^{b,c,1}, Yin Cai^{b,c}, Changhui Wang^{b,c}, Jin Wei^{b,c},
Ping Chen^{c,*,*}, Lei Shi^d, Jiangxing Wu^{a,c}

^a School of Computer Science and Artificial Intelligence, Zhengzhou University, Zhengzhou, China

^b School of Computer Science, Fudan University, Shanghai, China

^c Institute of Big Data, Fudan University, Shanghai, China

^d School of Cyber Science and Engineering, Zhengzhou University, Zhengzhou, China

ARTICLE INFO

Dataset link: <https://github.com/GuanhangShi/FDU/IDOR-GUARD>

Keywords:

Insecure direct object reference
Broken object-level authorization
Data-access mediation
Runtime authorization enforcement
Cross-language authorization

ABSTRACT

Insecure Direct Object Reference (IDOR), commonly characterized as Broken Object-Level Authorization (BOLA), remains a prevalent security risk in database-backed applications. The vulnerability arises when authenticated subject information is retained by the application layer, while a downstream ORM or SQL operation dereferences an attacker-controlled object identifier without verifying the corresponding subject-object authorization relation. Existing research primarily focuses on vulnerability detection and repair, leaving deployed legacy applications without a low-intrusion mechanism for runtime object-level authorization enforcement. This paper presents IDORacle, a template-guided data-access mediation framework for preventing horizontal privilege escalation at runtime. IDORacle propagates or recovers trusted subject context at an interceptable data-access boundary and evaluates protected operations against declarative authorization metadata. Its guard model supports direct ownership, derived ownership, group or tenant membership, role-sensitive state transitions, and sensitive-column policies, together with public-table and administrator-only exceptions. The complete Java reference implementation intercepts MyBatis/JDBC operations, compiles normalized SQL templates into reusable mediation plans, and combines runtime identity context, SQL abstract syntax trees, policy metadata, and cached authorization proofs to permit, rewrite, probe, block, or audit individual executions. Framework-native adapters instantiate the same subject-object mediation abstraction at Django ORM, SQLAlchemy-compatible, xorm, Laravel Eloquent, and CodeIgniter data-access boundaries. The evaluation comprises 22 paired authorization cases from eight applications across Java, Python, Go, and PHP. In the gray-box setting, every unmediated adversarial trace reaches a confirmed out-of-scope object and produces the prohibited database effect, yielding an ASR of 100%. With IDORacle enabled, the trusted subject is incorporated into the committing lookup, mutation, membership probe, or state-transition decision before the database effect is applied. Consequently, no guarded adversarial trace produces an unauthorized effect, yielding a residual ASR and FNR of 0%. Every paired authorized trace preserves its expected returned objects, affected-row count, and final state, yielding an FPR of 0%. In the Java microbenchmark, uncached guard processing averages 0.17 ms per instance and decreases to 0.017 ms for recurring templates with reuse, an approximate 10× reduction. Across the 17 plotted end-to-end request samples, reuse lowers the guarded mean latency relative to the corresponding no-reuse configuration. These results demonstrate that a common object-level authorization invariant can be enforced at heterogeneous ORM and SQL data-access boundaries while preserving framework-specific mediation mechanisms. The Java reference implementation, cross-language adapters, benchmark cases, and reproduction instructions are publicly available at <https://github.com/GuanhangShi/FDU/IDOR-GUARD>.

* Corresponding author.

E-mail addresses: sywt149@gs.zzu.edu.cn (Y. Song), guanhang@bu.edu (G. Shi), ycai25@m.fudan.edu.cn (Y. Cai), changhuiwang25@m.fudan.edu.cn (C. Wang), j_wei@fudan.edu.cn (J. Wei), pchen@fudan.edu.cn (P. Chen), shilei@zzu.edu.cn (L. Shi), jxwu@fudan.edu.cn (J. Wu).

¹ Yuewantong Song and Guanhang Shi contributed equally to this work.

1. Introduction

Broken access control remains one of the most persistent classes of web-application security failures. In the OWASP Top 10:2025, Broken Access Control retains its ranking as the leading application-security risk (OWASP Foundation, 2021, 2025; MITRE Corporation, 2026a). In the OWASP API Security Top 10 2023, Broken Object Level Authorization (BOLA) is listed as API1:2023, reflecting the broad attack surface created when APIs expose object identifiers through URLs, request bodies, and API parameters without enforcing object-level authorization checks (OWASP Foundation, 2023a). Insecure Direct Object Reference (IDOR) is a concrete instantiation of this problem: an authenticated user substitutes an object identifier — such as an order ID, job ID, tenant ID, or user ID — and the server omits verification that the referenced object falls within the user's authorized scope (MITRE Corporation, 2026b; PortSwigger Web Security Academy, 2026a,b).

Unlike injection vulnerabilities, IDOR is a semantic vulnerability. An HTTP request may be syntactically valid, the generated query may be well formed, and the authenticated user may be permitted to invoke the endpoint, yet the concrete database object accessed by the backend may belong to another user, tenant, or group. Correctness therefore depends on application-specific authorization invariants rather than on input syntax (Felmetsger et al., 2010; Pellegrino and Balzarotti, 2014; Wang et al., 2026). Object-level authorization cannot be addressed as a pure input-sanitization problem; it must be enforced at the layer where subjects, objects, and backend operations converge.

A substantial body of research has investigated static analysis, directed fuzzing, and runtime observation to detect BOLA and missing owner-check vulnerabilities in web applications (Huang et al., 2024; Liu et al., 2025a,b; Dharmadi et al., 2025; Zhang et al., 2025). These detection-oriented approaches effectively identify whether a vulnerable endpoint exists, yet they operate within a testing-phase paradigm. In enterprise environments, legacy codebases frequently contain large numbers of endpoints in which authorization logic is interleaved with business queries. Even when a scanner successfully flags a missing ownership check, retrofitting authorization across controllers, services, ORM helpers, and database mappings is error-prone and time-consuming. A deployable, low-intrusion runtime prevention mechanism is therefore needed as a complementary last line of defense.

A natural enforcement point is the data-access boundary. Prior work has explored fine-grained access control through query rewriting, authorization views, row-level security, and middleware mediation (Rizvi et al., 2004; Chaudhuri et al., 2007; Mehta et al., 2017; Eykholt et al., 2017; PostgreSQL Global Development Group, 2026; Microsoft, 2025; Zhang et al., 2022). These mechanisms establish that query mediation is both security-relevant and practical. Their direct deployment in existing application stacks nevertheless exposes a persistent semantic gap. Java applications may authenticate users in filters, annotations, controllers, or services before MyBatis/JDBC executes a resource-ID query. Python applications may expose the principal through `request.user` or an authenticated workspace context while Django QuerySets or SQLAlchemy-compatible statements construct the database predicate (Django Software Foundation, 2026b,a; SQLAlchemy Authors, 2026). Go applications may retain the subject in request context while xorm helpers select or update a record by an identifier (XORM Authors, 2026a,b). PHP applications similarly express persistence through Laravel Eloquent builders or CodeIgniter model and query-builder operations (Laravel LLC, 2026; CodeIgniter Foundation, 2026). Across these stacks, trusted subject evidence is separated from the operation that commits the object access.

This paper addresses this deployment gap by studying runtime prevention of horizontal object-level authorization violations in database-backed applications. The primary goal is not to replace framework-level authentication, RBAC, or ABAC (Sandhu et al., 1996; Hu et al., 2014), but to add a last-mile reference monitor that requires sufficient subject-object evidence before a protected data-access operation reaches the

database. The complete reference implementation is provided for the Java/MyBatis/JDBC stack, while framework-native adapters instantiate the same authorization relation through Django ORM, SQLAlchemy-compatible, xorm, Laravel Eloquent, and CodeIgniter query mechanisms (Django Software Foundation, 2026a; SQLAlchemy Authors, 2026; XORM Authors, 2026a; Laravel LLC, 2026; CodeIgniter Foundation, 2026).

Realizing this goal surfaces four challenges. First, authorization evidence is fragmented: ownership semantics range from direct owner columns to multi-hop relations, tenant or group membership, and state-dependent rules. Second, enforcement must be semantics-aware; uniformly appending `AND user_id = ?` would disrupt dictionary tables, administrator operations, and legitimate cross-user workflows. Third, the mediation abstraction must remain stable across heterogeneous query-construction mechanisms without claiming that one Java agent can be reused unchanged in every runtime. Fourth, the complete Java implementation must amortize repeated parsing and policy-binding costs in production services in which hot queries execute inside loops, scheduled jobs, and asynchronous tasks.

To address these challenges, we present **IDORacle**, a template-guided runtime prevention framework. (i) A common abstraction binds a trusted subject, protected object, operation, and policy-defined guard at an interceptable data-access boundary, subsuming the fragmented ownership evidence found in real applications. (ii) The guard model expresses authorization semantics through AST-level predicates, framework-native scopes, membership probes, and sensitive-column policies rather than a uniform blanket predicate, preserving legitimate dictionary, administrator, and cross-user workflows. (iii) The Java reference implementation places a mediation layer at MyBatis/JDBC sinks and propagates the authenticated subject through a server-side trace identifier; cross-language adapters translate the same guard semantics into the native query construction mechanism of each evaluated framework, without treating one Java agent as portable across runtimes. (iv) Template planning is separated from instance mediation: a SQL template is parsed, normalized, and compiled into a reusable mediation plan once, while per-execution work is limited to context binding, resource-key extraction, proof lookup, and guard evaluation.

This paper makes the following contributions:

- **Template-guided data-access reference monitor.** IDORacle introduces a low-intrusion reference monitor that binds trusted subject context to protected object operations. The complete Java realization separates template-level planning from instance-level mediation, reuses authorization plans across normalized SQL fingerprints, and enforces each runtime operation using trusted context and policy metadata.
- **Heterogeneous and portable guard model.** IDORacle defines guards for direct ownership, derived ownership, group or tenant membership, role or state policy, and sensitive-column policy. The same declarative semantics are instantiated through SQL AST constraints in the Java reference implementation and through framework-native query scoping in the evaluated Python, Go, and PHP adapters.
- **Multi-ecosystem evaluation.** We evaluate 22 paired authorization cases spanning eight real-world applications and four programming-language ecosystems: nine Java cases, six Python cases, two Go cases, and five PHP cases. In the unmediated gray-box baseline, every adversarial trace produces an unauthorized database effect. IDORacle prevents the prohibited effect in every guarded adversarial trace and preserves every paired authorized operation, corresponding to zero residual ASR, zero FNR, and zero FPR. The Java microbenchmark reduces mean per-instance guard processing from 0.17 ms without reuse to 0.017 ms for recurring templates, and reuse lowers the guarded mean latency in all 17 plotted end-to-end request samples.

- **Open-source artifact.** The complete Java reference implementation, cross-language adapter code, benchmark cases, experimental configurations, and reproduction instructions are publicly available at <https://github.com/GuanhangShiFDU/IDOR-GUARD>.

2. Background and related work

2.1. Object-level authorization in database-backed applications

Broken access control encompasses distinct authorization failures that require separate enforcement evidence. Broken Object-Level Authorization (BOLA), of which IDOR is a concrete manifestation, concerns whether an authenticated subject may access the particular object referenced by a request (OWASP Foundation, 2023a; MITRE Corporation, 2026b; PortSwigger Web Security Academy, 2026b). The endpoint itself may be legitimately callable, yet the selected order, job, tenant, or user record may fall outside the subject's authorized scope. Broken Function-Level Authorization instead concerns whether the subject may invoke a function or endpoint, whereas Broken Object Property-Level Authorization concerns access to individual fields of an otherwise accessible object (OWASP Foundation, 2023c,b). IDORacle therefore targets horizontal object-level violations as its primary case and handles function- or property-level restrictions only when the corresponding route, role, or column evidence can be carried to an interceptable data-access boundary.

Database-backed applications make this object-level decision difficult to enforce consistently because authorization evidence is distributed across architectural layers. A Java stack may authenticate a principal in a gateway, servlet filter, controller, or service before MyBatis/JDBC emits the concrete SQL statement. A Python application may expose the principal through `request.user` or a workspace context while Django QuerySets or SQLAlchemy-compatible statements construct the object predicate. A Go application may retain the subject in request context while xorm helpers select or update a record. PHP applications may carry identity through middleware or controllers before Eloquent repository or CodeIgniter model queries commit the operation (Django Software Foundation, 2026b,a; SQLAlchemy Authors, 2026; XORM Authors, 2026a; Laravel LLC, 2026; CodeIgniter Foundation, 2026). Connection pools and shared application credentials further prevent the DBMS from recovering the end-user identity by itself. Although the framework mechanisms differ, each stack exhibits the same last-mile separation between trusted subject evidence and the operation that dereferences or mutates the protected object.

To illustrate why this problem cannot be repaired by one universal predicate, consider two representative vulnerable paths:

```
// [Scenario A] Direct Ownership
Controller: requireLogin()
Mapper: SELECT * FROM t_order
        WHERE id = ?
// Missing: AND user_id = {curr_user}

// [Scenario B] Join-Derived Ownership
Controller: requireLogin()
Mapper: SELECT * FROM t_order_item
        WHERE item_id = ?
// Missing: ownership derived through t_order
// AND t_order.user_id = {curr_user}
```

In both cases, authentication succeeds and the SQL statement is syntactically valid, but the subject-object relation is not verified. Direct ownership may be expressed by an owner column, whereas other resources derive authorization through parent tables, tenant or group membership, or state-sensitive business rules. Conventional RBAC and ABAC provide essential role- and attribute-level policy models (Sandhu et al., 1996; Hu et al., 2014), but they do not by themselves guarantee that each concrete ORM or SQL operation is constrained to the current subject's object scope. This unresolved separation between trusted

identity, heterogeneous ownership evidence, and the committing data-access operation motivates a last-mile mechanism that re-associates the subject, object, and operation immediately before database execution.

2.2. Detection of logic and BOLA vulnerabilities

Prior research has established access-control defects as semantic web logic vulnerabilities rather than syntactic input-validation errors. Early systems such as NoTamper, MACE, and AuthScope reason about hidden parameters, session state, and protocol fields to expose parameter tampering and horizontal privilege escalation (Bisht et al., 2010; Monshizadeh et al., 2014; Zuo et al., 2017). Static and black-box analyses further show that authorization correctness depends on application-specific workflows, state transitions, and business invariants (Sun et al., 2011; Felmetzger et al., 2010; Pellegrino and Balzarotti, 2014; Wang et al., 2026). More recent systems improve reachability and semantic coverage through stateful crawling, evolutionary or directed fuzzing, multi-user differential execution, and runtime validation (Güler et al., 2024; Guo et al., 2025; Wang et al., 2025; Schlaubitz et al., 2025; Xu et al., 2025). These techniques substantially improve the ability to reach protected workflows and determine whether two users receive inconsistent authorization outcomes.

BOLA-specific studies narrow this reasoning to the relation between an authenticated principal, an attacker-controlled object reference, and the backend operation that dereferences it. BolaRay infers object-level authorization models in database-backed applications, while MOCGuard identifies missing owner checks in Java web applications (Huang et al., 2024; Liu et al., 2025a). BACScan and BACFuzz combine multi-user differential behavior, runtime evidence, and database observations to overcome the ambiguity of HTTP responses alone (Liu et al., 2025b; Dharmadi et al., 2025). UABScan exposes inconsistent authentication and authorization assumptions across Java web layers, and empirical studies further demonstrate that practical BOLA behavior extends beyond simple identifier guessing (Zhang et al., 2025; Wu et al., 2025; Kaur, 2026). Collectively, these systems clarify where an object-level check is absent and what ownership evidence is needed.

The remaining limitation is temporal and operational rather than diagnostic. Most of these approaches determine whether a vulnerable endpoint exists during testing, analysis, or repair, but do not continuously mediate the SQL operation when vulnerable legacy code remains deployed. Retrofitting owner checks across large services can also be incomplete because authorization logic is scattered across controllers, services, mappers, and asynchronous tasks. IDORacle is therefore complementary to detection and repair: inferred ownership relations can inform its policy metadata, while the framework enforces those relations at runtime before the corresponding SQL statement reaches the database.

2.3. Data-access authorization enforcement

Database and middleware research has long demonstrated that fine-grained data access can be enforced through authorization views, predicated grants, query rewriting, row-level security, and policy-aware mediation (Rizvi et al., 2004; Chaudhuri et al., 2007; PostgreSQL Global Development Group, 2026; Microsoft, 2025). Systems such as Qapla, Authorized Updates, and Blockaid further show that application queries can be checked or transformed outside ordinary business logic before reaching protected data (Mehta et al., 2017; Eykholt et al., 2017; Zhang et al., 2022). Related middleware and policy-recovery work improves scalable predicate application, query-control interfaces, and extraction of access-control rules from existing systems (Pappachan et al., 2020; Bogaerts et al., 2021; Shay et al., 2018; Zhang et al., 2023, 2024). These studies establish query mediation as a practical enforcement point and show that row visibility can be represented by policy-derived constraints.

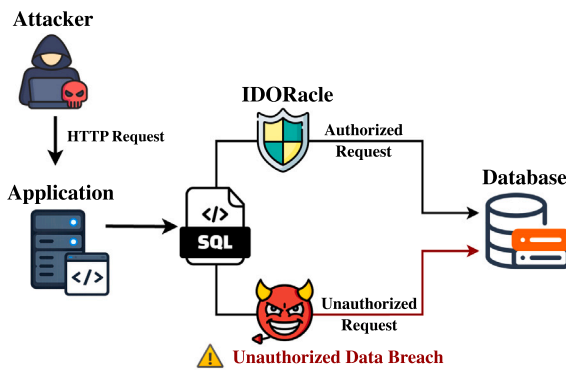


Fig. 1. Threat model of IDORacle.

Three deployment gaps remain across existing application stacks. First, database-native policies commonly operate on connection principals, whereas pooled connections conceal the authenticated application user associated with an individual query. Second, generic row predicates do not uniformly represent direct ownership, derived ownership, group membership, state transitions, and sensitive-column constraints. Third, data-access frameworks expose different stable mediation points: MyBatis/JDBC presents SQL templates and ASTs; Django ORM presents model QuerySets; SQLAlchemy-compatible persistence presents mapped statements and sessions; xorm exposes chainable helper conditions; and Laravel Eloquent and CodeIgniter expose model or query-builder operations (Django Software Foundation, 2026a; SQLAlchemy Authors, 2026; XORM Authors, 2026a; Laravel LLC, 2026; CodeIgniter Foundation, 2026). A practical runtime defense must preserve one subject-object policy abstraction while translating it into framework-native query semantics.

IDORacle addresses this gap by propagating or recovering trusted subject context at the data-access boundary, binding policy-defined guards to protected objects, and mediating each operation through allow, rewrite, probe, block, or audit decisions. The complete Java implementation compiles guards for normalized SQL templates and reuses them for hot executions. The cross-language adapters retain the same guard semantics but instantiate them through framework-native owner constraints. The corresponding attacker assumptions and trusted boundary are defined next, followed by the complete Java design and adapter mapping.

3. Threat model

Fig. 1 summarizes the attack path from an authenticated request to an unauthorized database effect and shows where IDORacle mediates the application-database interaction. The attacker and defender assumptions that make this mediation trustworthy are specified explicitly in Sections 3.1 and 3.2; they are not encoded as a separate trust-boundary region in the figure. In the cross-language adapters, the same subject-object check is placed at each framework's stable ORM, repository, model, or query-builder boundary rather than through the Java agent.

3.1. Attacker capabilities and objectives

As shown in Fig. 1, an authenticated low-privilege attacker manipulates request-controlled object identifiers that reach an insufficiently scoped ORM or SQL data-access operation. The adversary can send ordinary HTTP or RPC requests and freely modify request-controlled identifiers, including path variables, query parameters, request-body fields, and RPC arguments. These identifiers may later be bound to SQL statements as business keys. The adversary aims to read, update, delete, or trigger state changes on objects outside the subject's authorized

scope by exploiting missing owner checks, incomplete service validation, controller permission omissions, or inconsistent handling of group and tenant resources. The adversary does not possess database credentials, cannot directly modify policy metadata, and cannot forge the trusted authenticated identity supplied by the framework or server-side context channel.

3.2. Defender capabilities and assumptions

The defender controls the protected application deployment, the corresponding data-access adapter, and the authorization metadata used by IDORacle. The defender can recover authenticated subject context from the existing security framework, register model, table, column, relation, and operation policies, and deploy the mediator in shadow or enforce mode. The trusted computing base contains the identity extractor, context channel where applicable, framework adapter, metadata store, mediator, and database engine. The complete Java reference implementation additionally trusts the JVM, Java agent, MyBatis/JDBC hook, and SQL rewriter; the cross-language evaluations trust their respective authenticated request context and framework-native query-construction boundary. IDORacle assumes that upstream authentication and the selected mediation point remain uncompromised. Operations completed entirely inside opaque external services or before an instrumented data-access boundary are outside this threat model.

4. Methodology

4.1. Overview

As illustrated in Fig. 2, the architectural vulnerability emerges when an authenticated user, such as Alice, submits a data retrieval request triggering an underlying database query. Within the Vanilla SQL App execution path, fragmented authorization allows this request to bypass incomplete controller or service checks. Consequently, the raw SQL execution proceeds directly to the database without enforcing ownership constraints. This structural flaw leads to unauthorized database access, ultimately yielding an HTTP 200 OK response that exposes cross tenant data and signifies a successful IDOR attack.

For the identical input request, the IDORacle execution path applies a two-phase template-guided mediation pipeline. Phase 1 binds the TraceID security context and intercepts the query at the DAO/JDBC boundary to construct a guarded SQL candidate. Phase 2 evaluates the candidate through guard matching, runtime validation, and redundancy-aware reuse, then emits an allow, rewrite, block, or audit decision. In the illustrated cross-owner lookup, the secured query returns no matching row for the authenticated subject, leading the application to a not-found or empty-result path before unauthorized data is disclosed.

The IDORacle design does not replace existing framework-level authentication or role authorization. Instead, it treats these mechanisms as trusted evidence sources and revalidates the concrete object-level operation at the last stable data-access boundary. In the complete Java implementation this boundary is the MyBatis/JDBC SQL sink; in the cross-language implementations it is a framework-native QuerySet, mapped statement, xorm helper, Eloquent builder, or CodeIgniter model/query-builder operation.

4.2. Phase 1: Context-aware rewrite

The following two phases specify the complete Java reference implementation. The cross-language adapters preserve the context-binding, policy-selection, and decision semantics while replacing Java SQL-template operations with framework-native query objects, repository scopes, helper conditions, or model-level predicates.

Phase 1 establishes the trusted identity anchor for each SQL execution and compiles a reusable mediation plan before any concrete

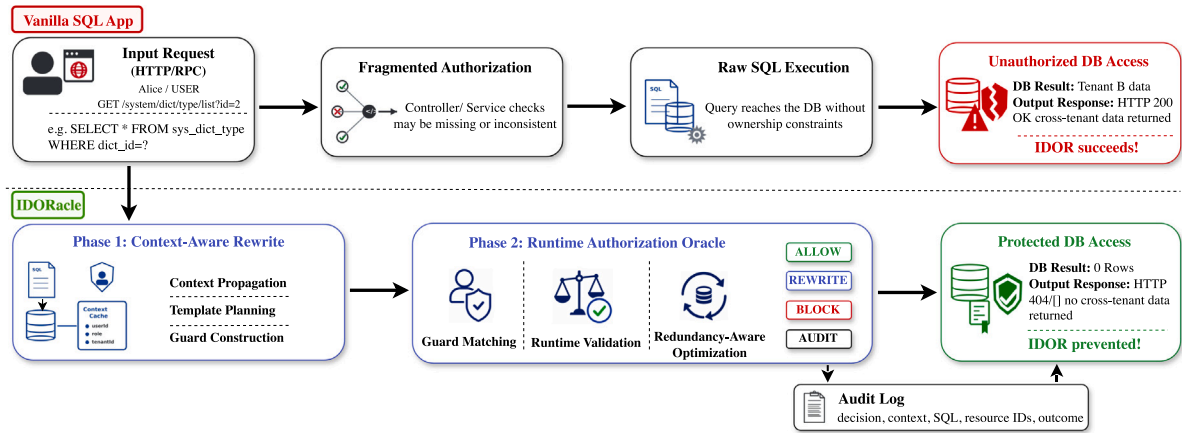


Fig. 2. Overview of the complete Java reference implementation of IDORacle, comparing the vanilla SQL execution path with template-guided mediation. Cross-language adapters instantiate the same subject-object mediation abstraction at framework-native data-access boundaries; SQL AST rewriting and proof caching remain properties of the Java implementation.

parameter values are bound. It comprises three sequential steps that correspond to the three operational lines shown in Fig. 2: binding the TraceID security context, intercepting the SQL at the Data Access Object (DAO)/JDBC boundary, and generating the secured SQL candidate that is forwarded to Phase 2 for evaluation.

Context propagation. A SQL instance can be mediated only if it is bound to the authenticated subject that triggered it. IDORacle therefore propagates a server-side *identity record*—referred to interchangeably as the *trusted subject context* throughout this paper—rather than trusting user-controlled request parameters. At the application entry point, the system extracts the authenticated principal and stores a compact identity record (Eq. (1)):

$$\text{ctx} : \{\text{traceId}\} \mapsto \{\text{userId, role, tenantId, sessionAttr, policyVersion}\}. \quad (1)$$

Only the identity record is propagated through Mapped Diagnostic Context (MDC), Remote Procedure Call (RPC) attachments, message headers, or scheduled-task wrappers to reliably bridge the semantic gap across thread boundaries; this follows the general practice of carrying opaque execution context rather than trusting user-supplied identity fields (World Wide Web Consortium, 2021). The full identity record remains in the trusted server-side store. At the SQL sink, the mediator first checks a short-lived local cache and then falls back to the shared context store. Missing or expired context is treated as a fail-closed condition unless the callsite is explicitly registered as a system task. This rule prevents silently allowing attacker-triggered SQL statements when context propagation fails.

Template planning. IDORacle separates SQL-template analysis from per-instance mediation. The SQL hook intercepts a parameterized SQL template at the DAO/JDBC boundary before concrete values are bound and constructs a template fingerprint h_T (Eq. (2)):

$$h_T = H(\text{mapperId, normalizedAst, commandType, parameterMappings, callsiteHash, policyVersion}). \quad (2)$$

Including the mapper identifier, parameter mapping, callsite hash, and policy version prevents unrelated SQL statements with similar Abstract Syntax Tree (AST) shapes from sharing an authorization plan. This strict fingerprinting reduces the risk that template plans are inadvertently shared across semantically distinct SQL statements or invalidated policy versions. If the fingerprint is unseen, the planner normalizes the SQL, parses its AST, extracts command type, tables, joins, subqueries, selected or updated columns, and predicate-related parameter positions, and then binds these facts to policy metadata. The output is a *TemplatePlan* (summarized in Table 1), which is cached locally and can be replicated to a distributed cache in multi-instance deployments.

Table 1
Main fields in a TemplatePlan.

Field	Description
templateHash	Fingerprint of the normalized SQL template and its execution context.
operation	SQL command type, such as SELECT, UPDATE, DELETE, or INSERT.
tables	Database tables touched by the SQL AST, including subqueries.
columns	Selected, updated, or predicate-related columns.
riskClass	Public table, direct ownership, derived ownership, group ownership, state transition, sensitive column, or admin-only access.
guards	Metadata-bound guards that must be evaluated for the template.
resourceKeys	Parameter positions or expressions used to derive protected resources.
action	Planned mediation action: allow, rewrite, probe, block, mask, or audit.

In the Java reference implementation, template planning is the main optimization for avoiding repeated SQL analysis. Java applications often execute a small number of SQL templates many times in loops, scheduled jobs, or branch-heavy business logic. IDORacle therefore performs expensive parsing, normalization, table classification, and policy binding once per template and keeps per-execution work limited to identity record binding, parameter extraction, proof lookup, and guard-specific mediation.

Guard construction. The planner attaches guards to a SQL template according to the metadata of the touched tables, columns, and operations. A guard describes how authorization should be established for a protected database object. To minimize deployment overhead, these guards and associated metadata are defined via a centralized, declarative configuration (e.g., YAML or JSON) maintained by the defender. This centralized configuration reduces duplication of the object-level guard logic mediated by IDORacle and provides one auditable policy source; it does not replace unrelated application-level authorization checks. Table 2 lists the guard types used by IDORacle. The Java implementation realizes these guards through AST predicates or proof queries, while the evaluated adapters map the same authorization relations to Django QuerySet filters, SQLAlchemy-compatible predicates, xorm owner conditions, Eloquent repository scopes, and CodeIgniter model or query-builder checks.

A table can carry multiple guards. For example, an order table may use direct ownership for ordinary user operations and an admin-only guard for back-office operations. A tenant-membership table may

Table 2
Guard types used for object-level data-access mediation.

Guard	Purpose
Direct ownership	Adds or verifies an owner predicate when the target table has an explicit owner column.
Derived ownership	Derives authorization through a parent table, foreign-key relation, or configured join path.
Group/tenant membership	Resolves group or tenant membership through a membership table or resource probe.
Role/state policy	Checks actor role, forbidden self-approval, or allowed state transition.
Sensitive-column policy	Blocks, masks, or audits unauthorized access to protected columns.
Public table	Allows explicitly registered dictionary or configuration tables.
Admin-only policy	Requires administrator or privileged-route evidence for protected operations.

Table 3
Examples of authorization metadata.

Object	Policy metadata
t_order	Direct owner column user_id.
t_order_item	Derived owner via order_id → t_order.id.
xxl_job_info	Group owner via job_group.
sys_user_tenant	Role/state guard for tenant approval.
sys_user.password	Sensitive column blocked for non-admin users.
t_sku_dict	Public dictionary table.

combine group membership and state-transition guards. A user table may combine direct ownership with sensitive-column mediation. Table 3 illustrates representative policy metadata entries. The guards are deliberately explicit: IDORacle does not infer ownership from arbitrary column names alone, because such inference would create both false positives and unsafe bypasses.

4.3. Phase 2: Runtime authorization oracle

Algorithm 1 formalizes the Java SQL-sink realization of the general mediation workflow.

Phase 2 evaluates the secured SQL candidate produced by Phase 1 against the runtime identity record and bound parameters, issuing one of four decisions: ALLOW, REWRITE, BLOCK, or AUDIT (shown in Fig. 2). The oracle comprises three coordinated sub-components — guard matching, runtime validation, and redundancy-aware optimization — formalized in Algorithm 1.

Guard matching. At runtime, the instance mediator evaluates the *TemplatePlan* against the current identity record and bound parameters. If the SQL template is registered as a public table access, the mediator issues an ALLOW decision immediately. If the template touches a protected table, the mediator extracts concrete resource keys according to the plan and evaluates the attached guards.

For direct ownership, the guard inserts a trusted owner predicate using the SQL AST rather than string concatenation:

```
SELECT * FROM t_order WHERE id = ?
⇒
SELECT * FROM t_order WHERE id = ? AND user_id = ?
```

The appended parameter is bound from $ctx:\{traceId\}$, not from the attacker-controlled request. UPDATE and DELETE statements are constrained in the same way so that unauthorized rows are not modified.

For derived ownership, the mediator uses metadata to connect a child table to an owner-bearing parent table. When safe SQL rewriting is possible — typically in single-table queries or straightforward joins — it inserts an EXISTS predicate:

```
SELECT * FROM t_order_item i WHERE i.id = ?
⇒
SELECT * FROM t_order_item i WHERE i.id = ?
AND EXISTS (SELECT 1 FROM t_order o
WHERE o.id = i.order_id AND o.user_id = ?)
```

However, AST rewriting can inadvertently alter query semantics. If the SQL dialect, complex aggregations (GROUP BY), unions (UNION), or certain Data Manipulation Language (DML) structures render rewriting unsafe, the mediator gracefully falls back to a proof query (PROBE-THEN-ALLOW) under the same request trace.

For group- or tenant-owned resources, the mediator first resolves the protected resource and then checks membership. A job operation, for example, may receive only job_id, while authorization is defined over job_group. The mediator resolves the group and verifies membership before the original SQL is permitted.

Runtime validation. Beyond ownership-derivable predicates, some cases require role- or state-sensitive checks that cannot be reduced to a row-level column condition. A tenant-join approval, for example, may require the actor to be an administrator of the tenant and not the applicant whose request is being approved. A user-list query may be legitimate for ordinary users but unsafe if it returns password hashes, tokens, phone numbers, or addresses. IDORacle handles these cases with role/state and sensitive-column guards.

A role/state guard checks the actor role, the protected resource, the old state, and the requested transition before the DML statement is executed. When the condition can be expressed safely in SQL, it is compiled into an EXISTS predicate. Otherwise, the mediator issues a proof query and blocks the update if the proof fails. A sensitive-column guard inspects the SELECT projection and applies the configured action: block, rewrite to NULL, mask the column, or audit. For example:

```
SELECT user_id, username, password, mobile FROM sys_user
⇒
SELECT user_id, username, NULL AS password,
CONCAT('***', RIGHT(mobile, 4)) AS mobile FROM sys_user
```

High-risk columns such as passwords and tokens are blocked by default unless route and role evidence explicitly authorize the access.

Redundancy-aware optimization. The result of each guard evaluation is cached as an authorization proof h_P (Eq. (3)):

$$h_P = H(\text{traceId}, \text{subjectHash}, \text{templateHash}, \text{operation}, \text{resourceKey}, \text{policyVersion}). \quad (3)$$

The authorization proof records the decision, evidence source, resolved resource attributes, guard type, and expiration time. To balance performance and consistency in concurrent environments, authorization proofs are assigned a short time-to-live (TTL). Write operations trigger a best-effort cache eviction for the affected resource keys, while policy-version changes immediately invalidate all stale authorization proofs. This mechanism is directly reflected in Algorithm 1 (lines 6–8), where a cache hit short-circuits repeated guard evaluation for hot SQL templates that recur in loops, paginated queries, or scheduled tasks.

4.4. Protected database access and enforcement outcomes

The oracle decision produced by Phase 2 determines the observable outcome at both the database layer and the application response layer, corresponding to the Protected DB Access box in Fig. 2. When the identity record is bound and the *TemplatePlan* carries a valid ownership-constrained SQL candidate, the database evaluates the rewritten statement and returns zero rows for unauthorized object references, compelling the application to return a safe HTTP 404 or an empty list response. No cross-tenant data is exposed, and the IDOR attempt is neutralized. Each decision is recorded in the audit log with the identity record, SQL fingerprint, guard type, and outcome timestamp.

Algorithm 1 SQL-Sink Mediation Logic**Require:** SQL template T ; bound parameters B ; identity record**Ensure:** Mediated SQL AST, or a BLOCK decision

```

1:  $C \leftarrow \text{LOADCONTEXT}(r)$ 
2: if  $C = \emptyset$  and  $\neg \text{ISYSTEMTASK}(T)$  then
3:   return BLOCK {fail closed}
4: end if
5:  $P \leftarrow \text{GETORCOMPILEPLAN}(T)$ 
6: if  $P.\text{riskClass} = \text{PUBLIC}$  then
7:   return ALLOW
8: end if
9:  $K \leftarrow \text{EXTRACTRESOURCEKEY}(B, P)$ 
10:  $h_p \leftarrow H(r, C, P, K)$ 
11: if  $\text{CACHEHAS}(h_p)$  then
12:   return  $\text{CACHEGET}(h_p)$ 
13: end if
14:  $D \leftarrow \text{EVALUATEGUARDS}(P.\text{guards}, C, K)$ 
15:  $\text{CACHEPUT}(h_p, D)$  {short TTL}
16: return  $D$ 

```

Operational modes. IDORacle supports two modes governing how oracle decisions are applied. In *shadow mode*, the framework records decisions and potential SQL differences without altering application behavior; this mode is used to validate metadata configurations, estimate false-positive rates, and identify legitimate cross-object queries that require explicit policy exceptions prior to production enforcement. In *enforce mode*, the oracle decision is applied before the SQL statement reaches the database. An ALLOW decision permits the original SQL to execute unmodified. A REWRITE decision substitutes the original AST with the ownership-constrained SQL candidate, producing the zero-row database result and safe HTTP response illustrated in Fig. 2. A BLOCK decision prevents database execution entirely. An AUDIT decision permits execution while recording a structured audit entry.

Fail-closed strategy. To ensure operational resilience, IDORacle adopts a fail-closed strategy for all critical error conditions. Missing identity record, absent policy metadata for protected tables, conflicting owner predicates, failed membership probes, and unauthorized sensitive-column accesses are all blocked by default rather than permitted, consistent with the fail-closed rule described in Section 4.2. The sole exception is a callsite that has been explicitly registered as a system task (e.g., scheduled jobs or migrations), in which case the absence of an end-user identity record does not by itself trigger blocking. This design guarantees that uncertainty in the authorization state is resolved conservatively, preventing attacker-triggered SQL statements from executing silently when context propagation fails or metadata is incomplete.

Explicit exception management. Explicit allow rules are required for public dictionary tables, migration tasks, scheduled system jobs, and administrator-only workflows. These exceptions are maintained in declarative metadata rather than in ad hoc code comments, ensuring that the policy configuration remains auditable and version-controlled. Original SQL comments are normalized and are not trusted as authorization evidence. When an audit comment is emitted, it carries only an opaque proof identifier; the corresponding evidence is stored exclusively in the trusted server-side channel, preventing comment-injection bypasses.

Framework-native guard instantiation. The cross-language adapters evaluated in Section 7.4 retain the same trusted-subject and ownership-policy model without reproducing every Java-specific optimization. The Django adapter converts a direct-owner guard into a user-scoped QuerySet condition; the Langflow adapter binds authenticated workspace or user context to SQLAlchemy-compatible lookups and mutations; the Gitea adapter adds owner-aware xorm conditions; and

the PHP adapters express the same relation through Eloquent repository scopes or CodeIgniter model and query-builder predicates. Cross-language generalizability is therefore evaluated at the level of authorization semantics, whereas template fingerprinting, AST rewriting, and authorization-proof caching remain properties of the complete Java reference implementation.

5. Benchmark design

The benchmark evaluates whether runtime data-access mediation prevents IDOR vulnerabilities when object-level authorization is missing, incomplete, or inconsistently placed across application layers. It combines the complete Java reference implementation with framework-native evaluation paths in Python, Go, and PHP. Each case follows the complete path from authenticated request to protected data effect and records the subject, attacker-controlled identifier, ORM or SQL operation, policy evidence, mediation decision, and resulting database or application outcome.

5.1. Design goals

The benchmark has three primary goals. First, it measures security effectiveness across direct ownership, derived ownership, group or tenant membership, collection scoping, object creation, and state-dependent authorization while separately checking preservation of authorized operations. Second, it evaluates cross-language transferability by instantiating the same subject-object guard abstraction at MyBatis/JDBC, Django ORM, SQLAlchemy-compatible, xorm, Laravel Eloquent, and CodeIgniter data-access boundaries. Third, it measures runtime practicality through repeated SQL templates, loop-generated mapper calls, and branch-heavy service paths in the complete Java reference implementation, and through representative end-to-end paths in the evaluated adapters. Performance comparisons remain within the same application and request path so that heterogeneous framework costs are not conflated.

5.2. Corpus selection, principals, and protected objects

The corpus is constructed as a purposive, reproducibility-oriented benchmark rather than a language-prevalence sample. A candidate case is retained when a public advisory identifies an object-level authorization failure, the affected version and patch evidence are available, the prohibited effect reaches an observable ORM or SQL data-access operation, and paired owner-authorized and cross-owner traces can be executed against a controlled fixture. Empirical work shows that vulnerability distributions vary across languages and projects and that limited project samples do not support language-level security rankings (Antal et al., 2020). The ecosystem composition is therefore driven by reproducible authorization evidence and diversity of data-access abstractions rather than by an assumed population proportion.

The Java suite contains nine cases and exercises the complete MyBatis/JDBC reference implementation. Seven cases are drawn from RuoYi: CVE-2025-28400, CVE-2025-28402, CVE-2025-28406, CVE-2025-28407, CVE-2025-28411, CVE-2025-28412, and CVE-2025-28413 (National Vulnerability Database, 2025a,b,c,d,e,f,g; CVE Public Advisory Repository, 2025; RuoYi Project, 2026). The remaining Java cases are XXL-Job CVE-2023-33779 and BootDo CNVD-C-2024-22302 (National Vulnerability Database, 2023; XXL-JOB Project, 2026; CT-FIOT Security, 2024; BootDo Project, 2026). Together, these cases cover direct and derived ownership, group-mediated authorization, collection scoping, and role-sensitive behavior.

The Python suite contains six cases. Two wger cases exercise an object-specific Django lookup and a collection QuerySet (National Vulnerability Database, 2026c,b; Django Software Foundation, 2026a). Four Langflow cases exercise owner-sensitive flow, API-key, and message operations at a SQLAlchemy-compatible persistence boundary

Table 4
Representative protected resources and authorization evidence for six applications.

Application	Vulnerability	Protected Resource	Authorization Evidence
RuoYi	CVE-2025-28407	sys_dict_type	Direct ownership through the create_by attribute
XXL-Job	CVE-2023-33779	xxl_job_info	Group ownership derived through job_group membership
BootDo	CNVD-C-2024-22302	sys_user	User-scoped access through the authenticated user_id
wger	CVE-2026-27839	nutrition_nutritionplan	Direct ownership through the user_id attribute
Gitea	CVE-2026-20904	user_open_id	Direct ownership through the authenticated uid
Krayin CRM	CVE-2026-38530	leads	Direct ownership through the lead's user_id attribute

(National Vulnerability Database, 2026e,d; CVE Program, 2026b; National Vulnerability Database, 2026i; SQLAlchemy Authors, 2026). The Go suite contains the two Gitea cases CVE-2026-27657 and CVE-2026-20904, whose email and OpenID operations are mediated through owner-aware xorm conditions (CVE Program, 2026a; National Vulnerability Database, 2026a; XORM Authors, 2026a,b).

The PHP suite contains five cases. Three Krayin CRM cases cover lead, person, and organization resources at an Eloquent-backed repository boundary (National Vulnerability Database, 2026f,g,j; Laravel LLC, 2026). Two Easy!Appointments cases cover appointment and provider authorization at a CodeIgniter model and query-builder boundary (National Vulnerability Database, 2026h; CodeIgniter Foundation, 2026). The Easy!Appointments record CVE-2026-52839 contributes two endpoint-distinct cases, one for creating an appointment under an out-of-scope provider and one for updating an existing appointment outside the authenticated user's authorization domain.

The resulting distribution is nine Java, six Python, two Go, and five PHP cases. The smaller Go subset follows from applying the same evidence requirements to the available candidates: fewer screened Go advisories simultaneously exposed a reproducible affected version, accessible patch evidence, an authorization effect at an interceptable data-access boundary, and a paired execution fixture. This selection outcome is not interpreted as a language-level prevalence or security ranking (Antal et al., 2020). In total, the benchmark contains 22 paired authorization cases associated with 21 public vulnerability records and spans eight applications. Table 4 presents six representative protected resources; the complete application-level outcomes are reported in Table 6.

The principal set contains at least two low-privilege users from different ownership domains and an administrator. This setup separates legitimate self-access, horizontal cross-user or cross-tenant access, and legitimate administrative access. Each protected object is associated with the trusted security-context field used as authorization evidence, such as userId, username, role, or tenantId.

5.3. Paired benign and adversarial traces

Each functional scenario is encoded as a pair of traces. The benign trace uses a resource identifier that belongs to the authenticated principal and must preserve the expected application behavior. The adversarial trace keeps the same principal and endpoint but changes the object identifier or transition target to a resource owned by another user, group, or tenant. This paired construction distinguishes successful attack prevention from indiscriminate denial of legitimate operations.

The representative Java overview case authenticates Alice and sends:

```
GET /system/dict/type/list?id=2
```

The vulnerable application emits:

```
SELECT * FROM sys_dict_type WHERE dict_id = ?
```

IDORacle adds trusted ownership evidence at the Java data-access boundary:

```
SELECT * FROM sys_dict_type WHERE dict_id = ?
AND create_by = ?
```

The same trace principle is applied to the Python, Go, and PHP suites. The wger and Langflow traces test object-specific and collection-level Python data access; the Gitea traces test owner-bound state-changing operations; and the Krayin CRM and Easy!Appointments traces test PHP repository, model, and query-builder paths. Security correctness is determined from returned objects, affected rows, inserted ownership relations, and final state; HTTP response classes are retained only as supporting evidence.

5.4. Case families

The benchmark contains four case families:

Object-level IDOR cases. These cases evaluate unauthorized reads, updates, deletes, and state changes triggered through attacker-controlled object identifiers. The expected protected behavior is either an owner-scoped operation that returns or affects no unauthorized object, or a BLOCK decision before execution.

Derived-ownership cases. These cases cover objects whose authorization evidence resides in a parent table, group, tenant, or configured resolver path. They prevent the benchmark from overfitting to a single owner = subject predicate.

Policy-boundary cases. These cases distinguish protected resources from legitimate public, administrative, or role-sensitive accesses. They test whether explicit policy exceptions preserve intended workflows rather than indiscriminately restricting every query.

Optimization stress cases. These Java cases include paginated list queries, loop-driven updates or deletes, and branch-heavy paths that repeatedly reach the same protected template. They measure whether template-plan reuse and authorization-proof caching reduce redundant work without changing security outcomes.

Mapping to the evaluated cases. Object-level IDOR is exercised across all four language ecosystems. Derived and group-based ownership is represented by the XXL-Job and RuoYi relation paths; collection scoping is represented by BootDo and wger; state-changing ownership checks are represented by Gitea, Krayin CRM, and Easy!Appointments; and framework-native object lookup is represented by Django ORM, SQLAlchemy-compatible persistence, xorm, Eloquent, and CodeIgniter. Optimization stress remains concentrated in the complete Java implementation, while the end-to-end measurements additionally characterize representative Python, Go, and PHP request paths.

5.5. Execution modes and measurements

The benchmark is executed in three modes that preserve the same application path and authorization policy while varying data-access mediation and decision reuse:

- **NO_GUARD:** the affected ORM or SQL operation executes without IDORacle, providing the vulnerable baseline for adversarial traces.
- **GUARD_NO_CACHE:** the configured guard is evaluated without reusable mediation decisions. In Java, this includes SQL normalization, guard matching, metadata lookup, ownership validation, and rewrite or probe construction; the cross-language adapters apply the corresponding framework-native owner or membership constraint.

Table 5
Implementation modules of the Java reference implementation.

Module	Technology	Responsibility
Guard Server	Spring Boot	Policy management and rewrite service
SQL Rewriter	JSqParser	SQL parsing and AST rewriting
Java Agent	Byte Buddy	Runtime SQL interception
Identity Layer	Servlet/Shiro Hooks	Identity extraction and propagation
Rewrite Cache	ConcurrentHashMap	Rewrite decision reuse
Observability UI	React + Vite	Visualization and benchmark control

- **GUARD_WITH_CACHE:** the same guard is enforced with reuse enabled. Java reuses normalized template plans and authorization proofs, while the evaluated adapter paths reuse the mediation decision supported by their framework-level implementation.

For every execution, the benchmark records the original data-access operation, the mediated operation or query when applicable, the authorization decision, affected rows or returned objects, response class, reuse status, and processing time. These records support the database-effect security oracle and the latency measurements reported in Section 7.

6. Implementation

IDORacle consists of a complete Java reference implementation and framework-native adapters used for cross-language evaluation. The Java prototype contains approximately 5.8 K lines of purpose-built code, excluding benchmark applications and dependencies: a backend policy engine of approximately 4.0 K lines for metadata management, decisions, and SQL rewriting, and a Java agent of approximately 1.8 K lines for JDBC interception, identity propagation, and transparent enforcement. The Python, Go, and PHP adapters instantiate the same trusted-subject and authorization-guard abstraction at their native data-access boundaries; they do not replicate every Java-specific template-planning and caching optimization (see Table 5).

The Java guard server is implemented in Spring Boot. It maintains the runtime switch, identity record, guard decisions, evaluation logs, and table-level ownership metadata. The metadata also supports administrator bypasses for cases where cross-user access is legitimate.

For SQL interception inside the Java evaluation testbed, IDORacle uses a MyBatis StatementHandler.prepare interceptor. Before SQL reaches the database, the interceptor obtains the current identity record, parses the SQL with JSqParser (JSqParser Project, 2026), detects the guarded table, and injects an ownership predicate into SELECT, UPDATE, and DELETE statements. For example:

```
// Original SQL
SELECT * FROM sys_notice WHERE notice_id = ?

// Rewritten guarded SQL
SELECT * FROM sys_notice WHERE notice_id = ?
AND create_by = currentUser
```

The injected value is derived from the trusted server-side identity record rather than from attacker-controlled request parameters. The external Java agent instruments JDBC prepareStatement calls in common connection implementations, including MySQL JDBC and Druid. It returns ALLOW, REWRITE, or DENY before the operation reaches the database. Framework-specific entry hooks for RuoYi, BootDo, and XXL-Job recover userId, username, and roles from Shiro or custom authentication filters without modifying legacy business code (RuoYi Project, 2026; BootDo Project, 2026; XXL-JOB Project, 2026).

Python adapters. The wger adapter recovers the trusted subject from request.user and registers owner metadata for the protected Django model (Django Software Foundation, 2026b). For CVE-2026-27839, an unscoped primary-key lookup is replaced by a QuerySet constrained by both the requested identifier and the current user. For CVE-2026-27835, collection QuerySets are constrained by the same authenticated owner instead of exposing another user's repetition configuration (National Vulnerability Database, 2026c,b; Django Software Foundation, 2026a).

The Langflow adapter obtains the authenticated workspace or user context before the persistence operation and binds that context to the requested flow, API-key, or message identifier. Langflow uses an internal persistence layer that is structurally compatible with SQLAlchemy but may not expose the full SQLAlchemy public API; accordingly, this paper uses the term *SQLAlchemy-compatible* to denote this boundary throughout. The four evaluated cases are mediated through SQLAlchemy-compatible filters or ownership probes before the protected object is returned, modified, or deleted (National Vulnerability Database, 2026e,d; CVE Program, 2026b; National Vulnerability Database, 2026i; SQLAlchemy Authors, 2026). Both Python realizations preserve the same direct-ownership or workspace-membership guard without relying on Java AST rewriting.

Gitea xorm adapter. Gitea uses an xorm-backed data-access layer in which chainable conditions can constrain record selection and updates (XORM Authors, 2026a,b). The Go adapter obtains the subject identifier from the authenticated request context and binds it to the record lookup or update helper. For CVE-2026-27657, the primary-email operation verifies that the selected email record belongs to the current user before it is made active. For CVE-2026-20904, the OpenID visibility update constrains the target record by both its identifier and owner UID (CVE Program, 2026a; National Vulnerability Database, 2026a; Gitea Project, 2026b,a). The guard is therefore realized as an owner-constrained xorm condition or helper parameter rather than Java AST rewriting.

PHP adapters. Krayin CRM uses an Eloquent-backed repository layer. The adapter binds the authenticated subject to lead, person, and organization queries so that a requested identifier must satisfy both the business lookup and the configured owner relation before the record can be returned or mutated (National Vulnerability Database, 2026f,g,j; Laravel LLC, 2026). Easy!Appointments uses a CodeIgniter model and query-builder boundary. Its adapter validates the appointment or provider relation before creating an appointment or updating an existing appointment (National Vulnerability Database, 2026h; CodeIgniter Foundation, 2026). These PHP realizations preserve the same subject-object invariant through framework-native predicates and ownership probes.

In Fig. 3, the lower SQL block represents the database-level constraint produced by mediation. For the Java cases, this is an AST-level SQL rewrite. For Django, xorm, and Eloquent cases, the equivalent predicate is introduced through the framework-native query or repository API before SQL execution. Thus, the figure compares authorization effects rather than implying that every adapter uses the Java SQL rewriter.

7. Evaluation

The deployment-time enforcement gap and heterogeneous mediation points identified in Section 2 motivate four evaluation dimensions: security effectiveness, practical deployment, cross-platform realization, and runtime performance. The evaluation addresses the following research questions:

- **RQ1 (Security Effectiveness):** How effectively does IDORacle prevent unauthorized object-level database effects while preserving authorized operations?

[Original SQL] <pre>UPDATE sys_dict_type SET dict_name = ? WHERE dict_id = ?</pre> [IDORacle-Rewritten SQL] <pre>UPDATE sys_dict_type SET dict_name = ? WHERE dict_id = ? AND create_by = :username</pre> (a) RuoYi (CVE-2025-28407)	[Original SQL] <pre>UPDATE xxl_job_info SET job_desc = ? WHERE id = ?</pre> [IDORacle-Rewritten SQL] <pre>UPDATE xxl_job_info SET job_desc = ? WHERE id = ? AND job_group IN :allowed_ids</pre> (b) XXL-Job (CVE-2023-33779)	[Original SQL] <pre>SELECT * FROM sys_user</pre> [IDORacle-Rewritten SQL] <pre>SELECT * FROM sys_user WHERE user_id = :user_id</pre> (c) BootDo (CNVD-C-2024-22302)
[Original SQL] <pre>SELECT * FROM nutrition_nutritionplan WHERE id = ?</pre> [IDORacle-Rewritten SQL] <pre>SELECT * FROM nutrition_nutritionplan WHERE id = ? AND user_id = :user_id</pre> (d) wger (CVE-2026-27839)	[Original SQL] <pre>UPDATE user_open_id SET show = NOT show WHERE id = ?</pre> [IDORacle-Rewritten SQL] <pre>UPDATE user_open_id SET show = NOT show WHERE id = ? AND uid = :user_id</pre> (e) Gitea (CVE-2026-20904)	[Original SQL] <pre>UPDATE leads SET title = ? WHERE id = ?</pre> [IDORacle-Rewritten SQL] <pre>UPDATE leads SET title = ? WHERE id = ? AND user_id = :user_id</pre> (f) Krayin CRM (CVE-2026-38530)

Fig. 3. Representative vulnerable data-access operations and framework-specific mediation effects for six applications selected from the eight-application evaluation corpus.

- **RQ2 (Practical Deployability):** Can IDORacle be integrated at existing data-access boundaries in real-world database-backed applications?
- **RQ3 (Cross-Platform Generalizability):** How consistently can the subject-object mediation abstraction be instantiated across heterogeneous frameworks and data-access mechanisms?
- **RQ4 (Runtime Performance):** What end-to-end latency impact does runtime mediation introduce across the evaluated application paths?

7.1. Experimental setup

Implementations and environment. The complete Java implementation provides authenticated-subject propagation, MyBatis/JDBC interception, SQL-template normalization, policy binding, AST-based rewriting, authorization probes, and reusable mediation decisions. The Java experiments use Spring Boot 2.7, MyBatis 3.5, MySQL 8.0, and HikariCP under a shared database-credential architecture. The experiments run on Ubuntu 20.04 LTS with a 3.6 GHz Intel Core i7 processor and 16 GB of memory.

The cross-language implementations mediate operations at the lowest stable framework boundary that jointly exposes the authenticated subject, target object, and pending data-access effect. The Python paths use Django QuerySets for wger and SQLAlchemy-compatible statements or sessions for Langflow. The Go path uses owner-constrained conditions in Gitea's xorm-backed layer. The PHP paths use Krayin CRM's Eloquent-backed repository operations and Easy!Appointments' CodeIgniter model or query-builder operations. These adapters instantiate the same trusted-subject and authorization relation through framework-native query representations; Java-specific template planning, AST rewriting, and authorization-proof caching remain properties of the complete Java implementation.

Evaluation corpus. The corpus contains 22 paired authorization cases from eight applications across four programming-language ecosystems. Each case contains one adversarial cross-subject trace and one corresponding authorized trace. The Java suite contains seven RuoYi cases, one XXL-Job case, and one BootDo case. The Python suite contains two wger cases and four Langflow cases. The Go suite contains two Gitea cases. The PHP suite contains three Krayin CRM cases and two Easy!Appointments cases. Because CVE-2026-52839 contributes two

distinct authorization paths, the 22 cases are associated with 21 public vulnerability records. The Java suite exercises the complete guard and optimization implementation, whereas the remaining suites evaluate transfer to distinct framework-native data-access abstractions.

Execution configurations. The experiments use three configurations:

- No GUARD executes the affected application path without IDORacle.
- GUARD No CACHE applies the configured authorization guard while disabling reusable mediation and authorization-proof decisions.
- GUARD WITH CACHE applies the same authorization guard with decision reuse enabled at the corresponding data-access boundary.

The two guarded configurations enforce identical authorization policies and differ only in whether previously computed mediation results are reused. The Java implementation reuses normalized template plans and authorization proofs; the evaluated adapter paths reuse their corresponding framework-level mediation decisions where reuse is implemented. Reuse changes the execution path but not the authorization predicate, evidence, or protected outcome.

Paired execution traces. For each case, the authorized trace operates on an object or collection within the authenticated subject's authorization domain, denoted by $A \rightarrow A$. The paired adversarial trace retains the same authenticated subject, endpoint, and business operation but supplies an object reference or transition target belonging to another user, group, or tenant, denoted by $A \rightarrow B$. This pairing tests whether the defense distinguishes ownership rather than indiscriminately denying the affected operation.

Each trace has a fixture-defined expected database effect. For an authorized trace, the expected effect specifies the objects that may be returned or modified, the affected-row count, and the relevant final database state. For an adversarial trace, the expected protected effect excludes every object outside the authenticated subject's authorization domain.

Metrics. Security outcomes are determined from the database effect rather than from the HTTP status code alone. An adversarial read succeeds when the returned result contains an out-of-scope object or protected field. An adversarial write succeeds when an out-of-scope row is inserted, updated, deleted, or moved to an unauthorized state. HTTP

Table 6

Effectiveness under the gray-box attack setting, where an authenticated attacker already holds a valid out-of-scope object identifier. The evaluation covers 22 paired authorization cases from eight applications across Java, Python, Go, and PHP. Each case contains one adversarial $A \rightarrow B$ trace and one authorized $A \rightarrow A$ trace. The no-cache and cached configurations are reported jointly because their security outcomes are identical.

Application	Adversarial $A \rightarrow B$			Authorized $A \rightarrow A$
	No Guard ASR (%)	IDORacle ASR (%)	IDORacle FNR (%)	IDORacle FPR (%)
RuoYi	100.00	0.00	0.00	0.00
XXL-Job	100.00	0.00	0.00	0.00
BootDo	100.00	0.00	0.00	0.00
wger	100.00	0.00	0.00	0.00
Langflow	100.00	0.00	0.00	0.00
Gitea	100.00	0.00	0.00	0.00
Krayin CRM	100.00	0.00	0.00	0.00
Easy!Appointments	100.00	0.00	0.00	0.00

responses are recorded as application-visible evidence because different frameworks may represent the same protected database outcome as an empty result, not-found response, or access denial.

The attack success rate (ASR) is the proportion of adversarial $A \rightarrow B$ traces that produce an unauthorized database effect. A guarded adversarial trace that still produces such an effect is a false negative; therefore, under this paired binary evaluation, the false-negative rate (FNR) is numerically equal to the residual ASR of IDORacle. Both metrics are reported separately because ASR characterizes the fraction of attacks that succeed from the attacker's perspective, whereas FNR characterizes the fraction of adversarial inputs that evade the defense from the defender's perspective; the two coincide only when each case contributes exactly one binary decision, as in the present benchmark. A false positive is recorded when IDORacle blocks or semantically changes an authorized $A \rightarrow A$ trace so that its returned objects, affected rows, or final state differ from the expected authorized outcome. The false-positive rate (FPR) is the proportion of authorized traces exhibiting this behavior.

Latency measurement. End-to-end latency includes request processing, application business logic, data-access mediation, database execution, and response construction. For every plotted Java, Python, Go, and PHP sample, each configuration is measured in five independent groups of 50 executions. Mean latency is computed over all 250 executions. The mean group maximum is the arithmetic mean of the five maximum values, one from each execution group. Figs. 4–7 report both statistics for RuoYi, wger, Krayin CRM, and Gitea.

Performance comparisons are conducted within the same application case and request path. The wger CVE-2026-27835 sample involves a collection-level QuerySet that spans two HTTP requests (list retrieval followed by item access); accordingly, the wger CVE-2026-27835 sample contains two HTTP requests, while the remaining plotted samples correspond to individual request paths.

7.2. RQ1: Security effectiveness

We evaluate security effectiveness under two threat settings. In the *gray-box* setting used in this section, an authenticated low-privilege attacker already holds a valid identifier for an object outside the authorized scope; each $A \rightarrow B$ trace therefore tests whether a confirmed foreign object reference can produce an unauthorized database effect through the affected application path. In the *black-box* setting analyzed in Appendix, the attacker must first discover a valid foreign identifier through bounded enumeration. The settings share the same authorization guards and differ only in whether identifier discovery is assumed. Table 4 presents representative authorization evidence for six applications, while Table 6 reports the complete eight-application corpus.

Unauthorized database effects. Table 6 reports an unmediated ASR of 100% for every application. This outcome follows from the gray-box construction rather than from a statistical extrapolation: each adversarial trace supplies a fixture-confirmed foreign object reference to an application path whose affected version omits the required object-level authorization relation. The database oracle consequently observes an out-of-scope read, write, deletion, creation, or state transition in every baseline case. The percentage characterizes the security effect after identifier acquisition; it does not represent the probability of discovering the identifier, which is analyzed separately in Appendix.

With IDORacle enabled, the residual ASR is 0% in every application because the trusted subject becomes part of the final authorization decision before the protected effect is committed. Direct-owner reads require the requested identifier and trusted owner predicate to match the same row. Updates and deletes apply the same conjunction to the affected-row set. Derived, group, tenant, and workspace relations are resolved through a configured join path or proof query. Collection operations are scoped to the authenticated principal, while object creation and state-transition paths validate the referenced owner, provider, or parent resource before insertion or mutation. These mechanisms differ across SQL ASTs, Django QuerySets, SQLAlchemy-compatible statements, xorm conditions, Eloquent scopes, and CodeIgniter model queries, but they enforce the same subject-object invariant. A foreign identifier may still be syntactically valid, yet it cannot satisfy the authorization evidence required by the committing data-access operation.

The cached and non-cached guarded configurations have identical security outcomes because reuse does not replace or weaken the authorization relation. A reusable decision is bound to the authenticated subject, protected resource, operation, and policy version. Evidence established for one subject, object, or policy version is therefore not treated as a generic permission for another execution.

False negatives and false positives. Under the paired metric definition, a guarded adversarial trace is a false negative precisely when an out-of-scope database effect remains observable. No guarded trace returns a foreign object, changes a foreign row, creates a relation under an unauthorized owner, deletes protected data, or completes a prohibited state transition. The FNR is therefore 0%, matching the residual ASR because each adversarial case contributes one binary security decision at the protected data-access boundary.

The FPR is also 0%, but it is established from a different evidence set. Every authorized trace traverses the same endpoint and business operation as its adversarial counterpart while supplying an object or relation within the principal's authorization domain. Authorized reads return the fixture-defined objects, permitted writes preserve the expected affected-row counts, authorized creations retain the intended owner or provider relation, and state-changing operations reach the expected final state. The absence of false positives is therefore attributable

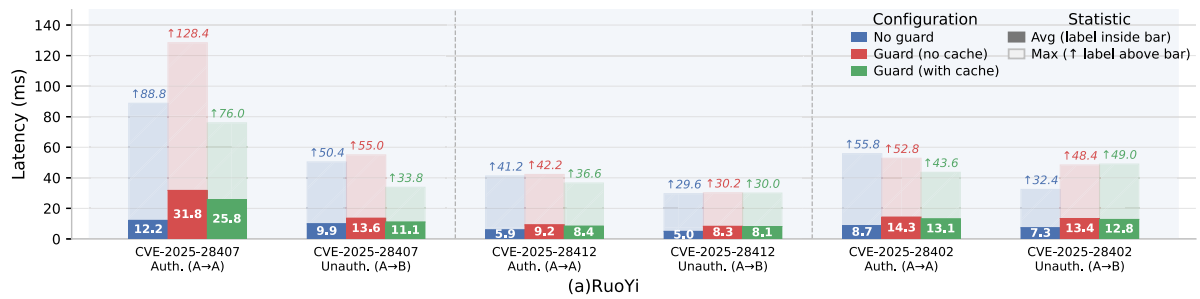


Fig. 4. End-to-end request latency for RuoYi. Bars report the mean results; labels above the bars report the arithmetic mean of the maxima.

to subject-sensitive discrimination rather than blanket denial of the vulnerable functionality.

Taken together, the ASR, FNR, and FPR results show that IDORacle changes the security-relevant database effect only when the requested object falls outside the authenticated subject's configured authorization domain. The result is consistent across all evaluated applications and guard realizations while remaining bounded to the authorization relations and mediation points exercised by the benchmark.

7.3. RQ2: Practical deployability

The eight evaluated applications expose distinct but usable mediation points. In RuoYi, BootDo, and XXL-Job, the complete Java implementation recovers the authenticated subject from Shiro or application-specific security contexts and propagates it to MyBatis/JDBC interception points, where the protected SQL template, bound identifier, and policy evidence are available together. Direct ownership can be compiled into an AST predicate, whereas group-derived or state-sensitive cases use configured relation paths or authorization probes.

The Python cases use two native representations. In wger, the Django adapter binds `request.user` to an object-specific lookup or collection QuerySet before query execution. In Langflow, authenticated workspace or user context is bound to SQLAlchemy-compatible flow, API-key, and message operations before the persistence effect. Gitea binds the request principal to xorm-backed lookup and update conditions. Krayin CRM applies owner-aware Eloquent repository scopes, while Easy!Appointments validates appointment and provider relations at its CodeIgniter model and query-builder boundary.

These placements share a practical property: the adapter is inserted at the lowest application-controlled boundary that simultaneously exposes trusted subject evidence, the protected object reference, and the pending database effect. Table 6 shows that every integration blocks cross-subject effects while preserving the corresponding owner-authorized behavior. The evidence covers scalar object reads, collection queries, updates, deletes, creation under a referenced owner, and owner-sensitive state transitions. Practical deployability is therefore supported across the evaluated framework boundaries without assuming that the implementation mechanism is identical across languages.

7.4. RQ3: Cross-platform generalizability

The authorization evidence exhibits a common invariant across the eight applications: a trusted authenticated subject is bound to the protected object and pending data-access operation before the database effect is committed. The frameworks differ in how these elements are represented and how the guard is materialized.

In Java, the subject is recovered from trace-bound server context, while the object and operation are extracted from the bound SQL instance and normalized AST. The complete implementation realizes direct, derived, group-based, and state-sensitive guards through predicate rewriting or authorization probes. In Django, the request

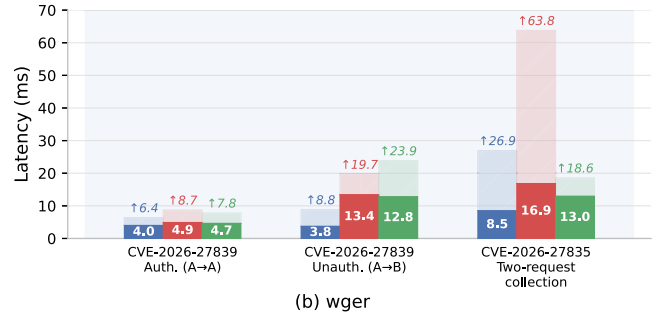


Fig. 5. End-to-end request latency for wger. Bars report the mean results; labels above the bars report the arithmetic mean of the maxima.

principal and model QuerySet expose subject and object at query-construction time. Langflow exposes mapped objects and statements through a SQLAlchemy-compatible persistence layer. Gitea combines request context with xorm helper arguments. Krayin CRM exposes Eloquent repository builders, and Easy!Appointments exposes CodeIgniter model and query-builder operations.

Every cross-language adversarial trace is mediated without returning or modifying an out-of-scope object, and every paired authorized trace retains its expected outcome. These results support portability of the subject-object authorization relation across SQL-AST constraints, ORM scopes, repository predicates, and proof queries. They do not imply implementation equivalence: the Python, Go, and PHP adapters instantiate the guards available at their framework boundaries, whereas the complete Java implementation additionally provides template fingerprinting, generalized AST rewriting, and authorization-proof caching.

7.5. RQ4: Runtime performance

Figs. 4–7 report the end-to-end latency of the evaluated Java, Python, PHP, and Go request paths under the three execution configurations. Each bar shows the mean over 250 executions, and each label above a bar shows the arithmetic mean of the five group maxima. The comparison for each sample is intra-path: it characterizes the cost of mediation and reuse for the same application behavior rather than comparing absolute latency across frameworks.

Java request paths. For the authorized RuoYi CVE-2025-28407 path, mean latency increases from 12.2 ms under No GUARD to 31.8 ms under GUARD No CACHE. Enabling reuse reduces the mean to 25.8 ms. The corresponding mean group maximum decreases from 128.4 to 76.0 ms between the two guarded configurations.

For CVE-2025-28412, the authorized-path mean changes from 5.9 ms to 9.2 ms and then 8.4 ms, while the mean group maximum changes from 41.2 ms to 42.2 ms and then 36.6 ms. For CVE-2025-28402, the authorized-path mean changes from 8.7 ms to 14.3 ms and

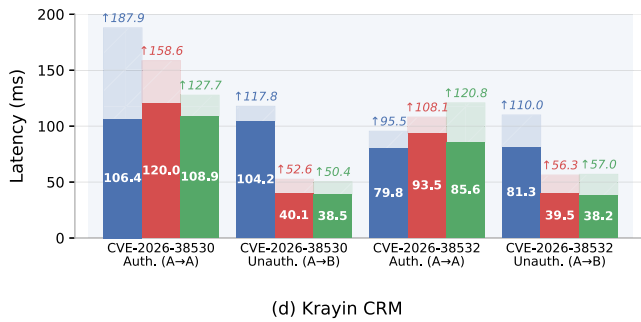


Fig. 6. End-to-end request latency for Krayin CRM. Bars report the mean results; labels above the bars report the arithmetic mean of the maxima.

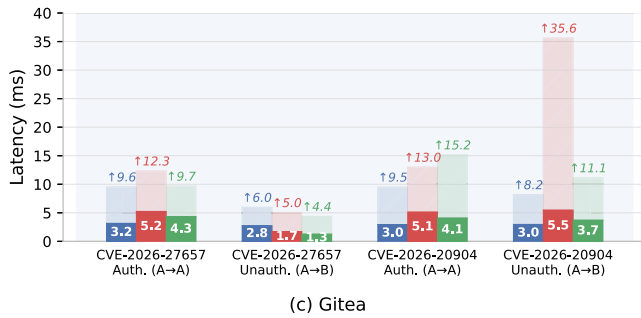


Fig. 7. End-to-end request latency for Gitea. Bars report the mean results; labels above the bars report the arithmetic mean of the maxima.

then 13.1 ms; the mean group maximum changes from 55.8 ms to 52.8 ms and then 43.6 ms. Reuse reduces the guarded mean in all three Java authorized paths and also lowers their mean group maximum.

The Java adversarial paths show the same reduction in mean guarded latency. For CVE-2025-28407, CVE-2025-28412, and CVE-2025-28402, the no-cache means of 13.6, 8.3, and 13.4 ms decrease to 11.1, 8.1, and 12.8 ms, respectively. Their mean group maxima change from 55.0 to 33.8 ms, 30.2 to 30.0 ms, and 48.4 to 49.0 ms. The final case therefore exhibits a lower mean but a slightly higher mean group maximum after reuse.

Python request paths. For the authorized wger CVE-2026-27839 object lookup, mean latency changes from 4.0 ms under No GUARD to 4.9 ms under GUARD No CACHE and 4.7 ms under GUARD WITH CACHE. Its mean group maximum changes from 6.4 to 8.7 and then 7.8 ms.

For the corresponding adversarial path, mean latency changes from 3.8 ms to 13.4 ms and then 12.8 ms. The owner-scoped lookup returns no matching object and is followed by the application's not-found response path. Its mean group maximum changes from 8.8 to 19.7 and then 23.9 ms, showing that the reduction in mean latency does not extend to the observed upper latency statistic in this case.

The CVE-2026-27835 collection sample contains two HTTP requests. Its mean latency changes from 8.5 ms to 16.9 ms and then 13.0 ms. The corresponding mean group maximum changes from 26.9 to 63.8 and then 18.6 ms. Decision reuse removes 3.9 ms from the guarded mean and substantially reduces the mean group maximum for this multi-request sample.

Go request paths. For the authorized Gitea CVE-2026-27657 path, mean latency changes from 3.2 ms to 5.2 ms and then 4.3 ms. Its mean group maximum changes from 9.6 to 12.3 and then 9.7 ms. For CVE-2026-20904, the authorized-path mean changes from 3.0 ms to 5.1 ms and then 4.1 ms, while the mean group maximum changes from 9.5 to 13.0 and then 15.2 ms.

The CVE-2026-27657 adversarial path terminates after the ownership condition rejects the selected email record. Its mean latency is

2.8 ms under No GUARD, 1.7 ms under GUARD No CACHE, and 1.3 ms under GUARD WITH CACHE. The corresponding mean group maxima are 6.0, 5.0, and 4.4 ms.

For the CVE-2026-20904 adversarial path, the guarded mean decreases from 5.5 to 3.7 ms after reuse, and the mean group maximum decreases from 35.6 to 11.1 ms. The resulting latency reflects the owner-constrained path, which prevents the unauthorized state-changing operation before completion.

PHP request paths. For the authorized Krayin CRM CVE-2026-38530 path, the mean latency is 106.4 ms without mediation, 120.0 ms with guard evaluation and no reuse, and 108.9 ms with reuse. The mean group maximum decreases from 187.9 ms in the baseline to 158.6 ms without reuse and 127.7 ms with reuse. For the authorized CVE-2026-38532 path, the corresponding means are 79.8, 93.5, and 85.6 ms. Reuse reduces the guarded mean, although the mean group maximum increases from 108.1 to 120.8 ms, indicating that the upper-tail statistic remains sensitive to endpoint-specific variability even when the average guarded path improves.

The two Krayin CRM adversarial paths terminate through owner-constrained control flow rather than completing the vulnerable mutation. For CVE-2026-38530, the mean decreases from 104.2 ms without mediation to 40.1 ms without reuse and 38.5 ms with reuse; the mean group maximum decreases from 117.8 to 52.6 and 50.4 ms. For CVE-2026-38532, the means are 81.3, 39.5, and 38.2 ms, while the mean group maxima are 110.0, 56.3, and 57.0 ms. These lower guarded adversarial latencies do not indicate that mediation reduces application cost; rather, the mediated path terminates before the vulnerable mutation completes, shortening the measured execution relative to the unprotected successful attack. The mediator changes the control flow by preventing the out-of-scope update before the vulnerable business operation completes, so the measured path is shorter than the unprotected successful attack.

Java guard-processing microbenchmark. The per-instance Java guard benchmark isolates mediation from the surrounding HTTP request lifecycle. Under GUARD No CACHE, complete normalization, classification, context binding, and rewrite or probe construction average 0.17 ms per execution. Under GUARD WITH CACHE, recurring templates reuse the compiled mediation plan and authorization proof, reducing the average to 0.017 ms per execution, approximately one tenth of the no-cache processing time. These measurements characterize the complete Java implementation and are not attributed to the framework-native adapters.

Overall performance. The reuse-enabled mean is lower than the corresponding no-reuse mean in all 17 plotted request samples across Ruoyi, wger, Gitea, and Krayin CRM. This consistency accords with the design of reuse: previously compiled mediation plans or framework-level decisions avoid repeated parsing, policy binding, or proof construction while retaining the same authorization evidence. The size of the reduction remains endpoint-dependent because mediation accounts for only one part of the request lifecycle.

The mean group maximum decreases in most, but not all, samples. The Java CVE-2025-28402 adversarial path, the wger CVE-2026-27839 adversarial path, the Gitea CVE-2026-20904 authorized path, and the Krayin CRM CVE-2026-38532 authorized path show that upper-tail observations can rise even when the guarded mean falls. The result distinguishes a stable reduction in average repeated work from a stronger claim about tail latency, which is influenced by database, framework, error-handling, and runtime scheduling variability.

Authorized paths provide the direct measure of mediation overhead because the same permitted business operation completes under all three configurations. Adversarial paths instead execute protected control flow: they may terminate before the vulnerable mutation, return an empty owner-scoped result, or invoke a framework-specific denial or not-found path. Their latency characterizes the cost of protected

execution and should not be treated as an overhead directly comparable with a successful unmediated attack. Overall, IDORacle introduces endpoint-dependent overhead on authorized operations, while reuse consistently reduces guarded mean latency across the evaluated request paths.

8. Discussion

8.1. Scope of data-access enforcement

IDORacle targets horizontal object-level authorization violations whose security effect reaches an interceptable ORM or SQL data-access operation. Its protection boundary is narrower than the full space of access-control failures. The complete Java implementation mediates MyBatis/JDBC operations, while the cross-language adapters instantiate guards at Django QuerySet, SQLAlchemy-compatible, xorm, Eloquent repository, and CodeIgniter model or query-builder boundaries. The system is applicable when the attacker-controlled identifier and sufficient ownership, membership, or state evidence are observable at the selected mediation point.

Several limitations remain. Operations completed through opaque third-party APIs without a locally observable data-access event evade interception. Stored procedures or generated queries that conceal resource semantics may require manual adapters. Authorization rules based exclusively on volatile business state that is absent from the identity record and policy metadata cannot be inferred by a data-access monitor alone. Highly dynamic ORM frameworks may also reduce the stability of raw SQL fingerprints; in such settings, mediation can be placed at a stable framework abstraction such as Hibernate query APIs, Django QuerySets, SQLAlchemy statements, GORM scopes, Eloquent builders, or CodeIgniter model queries (Hibernate Team, 2026; Django Software Foundation, 2026a; SQLAlchemy Authors, 2026; GORM Project, 2026; Laravel LLC, 2026; CodeIgniter Foundation, 2026).

A broader avenue for future work is integration with static analysis or authorization-inference tools such as BolaRay or MOCGuard (Huang et al., 2024; Liu et al., 2025a). Automatically generating baseline policy templates from inferred ownership relations would reduce manual configuration overhead and allow IDORacle to act as the runtime enforcement engine for statically derived policies.

8.2. Layer placement and deployment trade-offs

IDORacle does not require developers to rewrite every controller, service, or mapper. The complete Java deployment enforces constraints at the DAO/JDBC boundary, while the cross-language adapters use framework-native data-access points. Lower-layer placement offers broad coverage and protects the database even when upstream checks are absent, but a malicious request may consume routing, parameter-parsing, and business-logic resources before it is rejected. Higher ORM or repository placement exposes more stable model semantics but requires framework-specific integration. IDORacle therefore complements rather than replaces early-stage RBAC, ABAC, and route authorization.

Limitations and future work. Several directions remain for future investigation. First, extending intra-service data-access mediation to microservice and distributed architectures remains an important direction. Future work can investigate trace-based subject propagation together with distributed policy and proof caches to preserve authorization context across service invocation chains and transaction boundaries, while measuring the resulting consistency and latency costs.

Second, the ecosystem sample is purposive rather than population-proportional. The nine Java cases exercise the complete implementation and heterogeneous guard types; the six Python, two Go, and five PHP cases provide transfer evidence at several framework-native

boundaries. The smaller Go subset reflects the number of screened candidates that satisfied the common reproducibility, patch, observable-effect, and paired-fixture requirements, not an estimate of language-level IDOR prevalence or framework security (Antal et al., 2020). Broader evaluation should extend the same invariant to additional SQL dialects and query abstractions, including Hibernate, SQLAlchemy variants, GORM, Laravel Eloquent, and other CodeIgniter applications, while preserving framework-specific mediation at each stable query boundary.

Finally, the current study combines a complete Java reference implementation with lightweight cross-language adapters in controlled environments. Large-scale production evaluation should use mirrored traffic, swimlane-based gray deployment, and controlled comparative validation to measure end-to-end latency, benign-request preservation, policy drift, and cross-service context consistency under highly concurrent workloads.

9. Conclusion

This paper has presented IDORacle, a template-guided runtime prevention framework for object-level authorization violations in database-backed applications. The complete Java/MyBatis/JDBC implementation binds trusted subject context to SQL instances and realizes direct, derived, group, role or state, and sensitive-column guards through template planning, AST rewriting, authorization probes, and reusable proofs. Framework-native adapters instantiate the same subject-object invariant through Django QuerySet scoping, SQLAlchemy-compatible predicates, owner-constrained xorm conditions, Eloquent repository scopes, and CodeIgniter model or query-builder checks.

The evaluation comprises 22 paired authorization cases from eight applications across Java, Python, Go, and PHP. Every unmediated gray-box adversarial trace produces its prohibited database effect, whereas IDORacle prevents every guarded cross-subject effect. This yields an ASR and FNR of 0% because no foreign read, write, deletion, creation, or state transition remains observable after mediation. Every paired authorized trace preserves its expected objects, affected-row count, ownership relation, and final state, yielding an FPR of 0% and showing that the result is not produced by indiscriminate denial. In the Java microbenchmark, reuse reduces mean per-instance guard processing from 0.17 ms to 0.017 ms. Across the 17 plotted end-to-end request samples, reuse lowers guarded mean latency in every case, while the magnitude of overhead and the mean group maxima remain endpoint-dependent. The results demonstrate that a common object-level authorization invariant can be enforced at heterogeneous ORM and SQL data-access boundaries without erasing the implementation differences among the participating frameworks.

CRedit authorship contribution statement

Yuewanton Song: Writing – review & editing, Writing – original draft. **Guanhang Shi:** Software. **Yin Cai:** Validation. **Changhui Wang:** Investigation. **Jin Wei:** Supervision. **Ping Chen:** Project administration, Conceptualization. **Lei Shi:** Supervision. **Jiangxing Wu:** Supervision.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix. Bounded black-box object-ID enumeration

The direct experiment in Section 7.2 measures authorization enforcement after the attacker has obtained a valid out-of-scope identifier. This appendix considers object-ID enumeration in which the identifier is initially unknown and the attacker performs a bounded number of non-repeated guesses. The calculation applies only to cases whose attack surface can be represented as a scalar candidate identifier space.

A.1. Attack model

For an eligible scalar object-reference case c , let K_c denote the candidate identifier space after identifiers known to belong to the attacker have been excluded. Let E_c denote the number of deleted or nonexistent identifiers in this space. The remaining $K_c - E_c$ identifiers correspond to valid objects outside the attacker's authorization domain.

For a maximum guess budget g , the number of attempted identifiers is

$$q_c(g) = \min(g, K_c). \quad (\text{A.1})$$

The guesses are sampled uniformly without replacement.

Let $\delta_{m,c}$ represent the database-effect outcome after a valid foreign identifier is reached:

$$\delta_{m,c} = \begin{cases} 1, & \text{if configuration } m \text{ produces an unauthorized} \\ & \text{database effect for case } c, \\ 0, & \text{otherwise.} \end{cases} \quad (\text{A.2})$$

The direct execution trace determines $\delta_{m,c}$. For the affected application without mediation, $\delta_{0,c} = 1$. For IDORacle, $\delta_{1,c} = 0$ for every case in Table 6.

Targeted attack. A targeted attacker preselects one specific victim object. The probability of reaching that object within $q_c(g)$ non-repeated guesses is $q_c(g)/K_c$. The targeted attack success rate is

$$\text{T-ASR}_m(c, g) = \delta_{m,c} \frac{q_c(g)}{K_c}. \quad (\text{A.3})$$

Untargeted attack. An untargeted attacker succeeds after reaching any valid foreign object. Failure occurs when every attempted identifier belongs to the E_c empty positions. The untargeted attack success rate is

$$\text{U-ASR}_m(c, g) = \delta_{m,c} \left[1 - \frac{\binom{E_c}{q_c(g)}}{\binom{K_c}{q_c(g)}} \right], \quad (\text{A.4})$$

where the binomial ratio is defined as zero when $q_c(g) > E_c$.

The targeted metric requires one preselected victim object, whereas the untargeted metric accepts any valid object outside the authenticated subject's authorization domain. Consequently,

$$\text{U-ASR}_m(c, g) \geq \text{T-ASR}_m(c, g). \quad (\text{A.5})$$

Three benchmark cases are not represented in Table A.7 because their attack surfaces do not satisfy the scalar object-reference model. BootDo CNVD-C-2024-22302 and wger CVE-2026-27835 expose collection-level access rather than one attacker-guessed foreign object identifier. The Easy!Appointments CVE-2026-52839 appointments/store case creates a new appointment under an out-of-scope provider and therefore validates a referenced owner relation rather than dereferencing an existing protected appointment identifier. The endpoint-distinct appointments/update case from the same vulnerability record does use an existing appointment identifier and is included. Consequently, the black-box calculation covers 19 scalar cases from the 22-case benchmark without changing the gray-box effectiveness corpus.

A.2. Enumeration results

Table A.7 shows that targeted success varies with the size of the candidate identifier space. Under five guesses, the vulnerable T-ASR ranges from 0.13% to 8.47%; under 50 guesses, it ranges from 1.26% to 84.75%. Untargeted success is substantially higher because every valid out-of-scope identifier satisfies the attack objective. In the reported fixtures, the vulnerable U-ASR is at least 99.97% under five guesses and rounds to 100.00% under 50 guesses.

For IDORacle, the database-effect indicator is zero for every reported scalar case because a discovered foreign identifier still fails the required subject-object guard. Both residual T-ASR and U-ASR are therefore 0% under the two guess budgets. The defense does not rely on making identifiers undiscoverable; it prevents a discovered out-of-scope identifier from producing the prohibited database effect.

A.3. Worked example: wger CVE-2026-27839

The wger fixture assigns two nutrition plans to each user. Across 200 historical users, the fixture allocates

$$200 \times 2 = 400 \quad (\text{A.6})$$

plan identifiers. Forty users and their corresponding 80 plans have been deleted, and plan identifiers are not reused. Alice remains active and knows the identifiers of her own two plans; these identifiers are removed from the attack space.

The remaining 159 active users own

$$159 \times 2 = 318 \quad (\text{A.7})$$

valid plans outside Alice's authorization domain. The candidate space therefore contains

$$K = 318 + 80 = 398 \quad (\text{A.8})$$

identifiers, comprising 318 valid foreign plans and $E = 80$ deleted or nonexistent positions.

Targeted enumeration. Suppose Alice selects one specific nutrition plan belonging to Bob but does not know its identifier. The target occupies one position in the 398-element candidate space. Under five non-repeated guesses,

$$\begin{aligned} \text{T-ASR}_0 @ 5 &= \frac{5}{398} \\ &= 1.2563\% \approx 1.26\%. \end{aligned} \quad (\text{A.9})$$

Under 50 guesses,

$$\begin{aligned} \text{T-ASR}_0 @ 50 &= \frac{50}{398} \\ &= 12.5628\% \approx 12.56\%. \end{aligned} \quad (\text{A.10})$$

Untargeted enumeration. For an untargeted attack, any of the 318 valid foreign plans satisfies the attack objective. Five guesses all fail only when all selected identifiers belong to the 80 empty positions. Therefore,

$$\begin{aligned} \text{U-ASR}_0 @ 5 &= 1 - \frac{\binom{80}{5}}{\binom{398}{5}} \\ &= 99.9703753\% \approx 99.97\%. \end{aligned} \quad (\text{A.11})$$

For 50 guesses, the all-miss probability is

$$\frac{\binom{80}{50}}{\binom{398}{50}} \approx 6.804 \times 10^{-43}. \quad (\text{A.12})$$

Expressed as a percentage, this probability is approximately $6.804 \times 10^{-41}\%$. Hence,

$$\text{U-ASR}_0 @ 50 \approx 100.00\%. \quad (\text{A.13})$$

The value reported in Table A.7 is rounded to two decimal places.

Table A.7

Effectiveness under the black-box attack setting with bounded object-ID enumeration across 19 scalar object-reference cases. Each cell reports *Vulnerable*→*IDORacle* (%). T-ASR and U-ASR denote targeted and untargeted attack success rates, respectively.

Application	CVEs	Protected endpoint	Guess attempt $g = 5$		Guess attempt $g = 50$	
			T-ASR (%)	U-ASR (%)	T-ASR (%)	U-ASR (%)
RuoYi	CVE-2025-28407	dictId	0.84 → 0.00	99.97 → 0.00	8.38 → 0.00	100.00 → 0.00
	CVE-2025-28413	editSave	0.84 → 0.00	99.97 → 0.00	8.38 → 0.00	100.00 → 0.00
	CVE-2025-28412	noticeId	1.26 → 0.00	99.97 → 0.00	12.56 → 0.00	100.00 → 0.00
	CVE-2025-28402	jobId	0.50 → 0.00	99.97 → 0.00	5.03 → 0.00	100.00 → 0.00
	CVE-2025-28406	jobLogId	0.13 → 0.00	99.97 → 0.00	1.26 → 0.00	100.00 → 0.00
	CVE-2025-28411	tableId	1.26 → 0.00	99.97 → 0.00	12.56 → 0.00	100.00 → 0.00
	CVE-2025-28400	postId	1.26 → 0.00	99.97 → 0.00	12.56 → 0.00	100.00 → 0.00
XXL-Job	CVE-2023-33779	jobId	0.13 → 0.00	99.97 → 0.00	1.26 → 0.00	100.00 → 0.00
wger	CVE-2026-27839	planId	1.26 → 0.00	99.97 → 0.00	12.56 → 0.00	100.00 → 0.00
Langflow	CVE-2026-34046	flow_id	1.26 → 0.00	99.97 → 0.00	12.56 → 0.00	100.00 → 0.00
	CVE-2026-33053	api_key_id	1.26 → 0.00	99.97 → 0.00	12.56 → 0.00	100.00 → 0.00
	CVE-2026-33760	message_id	0.13 → 0.00	99.97 → 0.00	1.26 → 0.00	100.00 → 0.00
	CVE-2026-55255	flow_id	1.26 → 0.00	99.97 → 0.00	12.56 → 0.00	100.00 → 0.00
Gitea	CVE-2026-27657	email_address.id	1.67 → 0.00	99.97 → 0.00	16.72 → 0.00	100.00 → 0.00
	CVE-2026-20904	user_open_id.id	8.47 → 0.00	99.98 → 0.00	84.75 → 0.00	100.00 → 0.00
Krayin CRM	CVE-2026-38530	leadId	0.50 → 0.00	99.97 → 0.00	5.03 → 0.00	100.00 → 0.00
	CVE-2026-38532	personId	0.84 → 0.00	99.97 → 0.00	8.38 → 0.00	100.00 → 0.00
	CVE-2026-61460	organizationId	1.26 → 0.00	99.97 → 0.00	12.56 → 0.00	100.00 → 0.00
Easy Appointments	CVE-2026-52839	appointmentId	0.13 → 0.00	99.97 → 0.00	1.26 → 0.00	100.00 → 0.00

Owner-scoped mediation. The affected Django lookup resolves a nutrition plan using only its request-controlled primary key:

```
NutritionPlan.objects.get(pk=plan_id)
```

The guarded lookup binds the same object reference to the authenticated subject:

```
NutritionPlan.objects.get(
    pk=plan_id,
    user=request.user)
```

The resulting database condition is equivalent to

```
SELECT ...
FROM nutrition_nutritionplan
WHERE id = ?
AND user_id = alice_user_id
```

A plan owned by Bob does not satisfy Alice's owner predicate even when its identifier is guessed correctly. Thus,

$$\delta_{l,c} = 0, \quad (\text{A.14})$$

and

$$T-ASR_l @ 5 = U-ASR_l @ 5 = T-ASR_l @ 50 = U-ASR_l @ 50 = 0\%. \quad (\text{A.15})$$

The candidate-space composition determines the probability of reaching a valid foreign plan, while the owner-scoped condition determines whether that plan can produce an unauthorized database effect.

Data availability

The source code and supporting materials are publicly available at <https://github.com/GuanhangShiFDU/IDOR-GUARD>.

References

- Antal, G., Mosolygó, B., Vándor, N., Hegedűs, P., 2020. A data-mining based study of security vulnerability types and their mitigation in different languages. In: Computational Science and Its Applications – ICCSA 2020. In: Lecture Notes in Computer Science, vol. 12252, Springer, pp. 1019–1034. http://dx.doi.org/10.1007/978-3-030-58811-3_72.
- Bisht, P., Hinrichs, T., Skrupsky, N., Bobrowicz, R., Venkatakrishnan, V.N., 2010. NoTammer: Automatic blackbox detection of parameter tampering opportunities in web applications. In: Proceedings of the 17th ACM Conference on Computer and Communications Security. CCS '10, Association for Computing Machinery, New York, NY, USA, pp. 607–618. <http://dx.doi.org/10.1145/1866307.1866375>.
- Bogaerts, J., Lagaisse, B., Joosen, W., 2021. SEQUOIA: A middleware supporting policy-based access control for search and aggregation in data-driven applications. IEEE Trans. Dependable Secur. Comput. 18 (1), 325–339. <http://dx.doi.org/10.1109/TDSC.2018.2889309>.
- BootDo Project, 2026. BootDo: An open-source java EE framework based on spring boot and mybatis. URL: <https://github.com/lcg0124/bootdo>, (Accessed 20 June 2026), GitHub repository.
- Chaudhuri, S., Dutta, T., Sudarshan, S., 2007. Fine grained authorization through predicated grants. In: Proceedings of the 23rd IEEE International Conference on Data Engineering. ICDE '07, IEEE, pp. 1174–1183. <http://dx.doi.org/10.1109/ICDE.2007.368972>.
- CodeIgniter Foundation, 2026. Query builder class. URL: https://www.codeigniter.com/userguide3/database/query_builder.html, Official CodeIgniter documentation; (Accessed 07 August 2026).
- CTFIOT Security, 2024. BootDo framework vulnerability analysis: Unauthorized access, sql injection, information disclosure, and stored XSS. URL: <https://www.ctfiot.com/206530.html>, (Accessed 20 June 2026).
- CVE Program, 2026a. CVE-2026-27657 Record. URL: <https://www.cve.org/CVERecord?id=CVE-2026-27657>, (Accessed 03 August 2026).
- CVE Program, 2026b. CVE-2026-33760 Record. URL: <https://www.cve.org/CVERecord?id=CVE-2026-33760>, (Accessed 07 August 2026).
- CVE Public Advisory Repository, 2025. Public RuoYi CVE Advisory Cases. URL: https://github.com/20210607/cve_public/tree/main/ruoyi_case, (Accessed 20 June 2026), GitHub repository.
- Dharmaadi, I.P.A., Alhanahnah, M., Pham, V.-T., Mohsen, F., Turkmen, F., 2025. BACFuzz: Exposing the silence on broken access control vulnerabilities in web applications. <http://dx.doi.org/10.48550/arXiv.2507.15984>, arXiv:2507.15984.
- Django Software Foundation, 2026a. QuerySet API reference. URL: <https://docs.djangoproject.com/en/5.2/ref/models/querysets/>, Django 5.2 documentation; accessed 2026-08-03.
- Django Software Foundation, 2026b. Using the django authentication system. URL: <https://docs.djangoproject.com/en/5.2/topics/auth/default/>, Django 5.2 documentation; accessed 2026-08-03.
- Eykholt, K., Prakash, A., Mozafari, B., 2017. Ensuring authorized updates in multi-user database-backed applications. In: 26th USENIX Security Symposium. In: USENIX Security '17, USENIX Association, Vancouver, BC, Canada, pp. 1445–1462, URL: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/eykholt>.

- Felmetsger, V., Cavedon, L., Kruegel, C., Vigna, G., 2010. Toward automated detection of logic vulnerabilities in web applications. In: Proceedings of the 19th USENIX Security Symposium. In: USENIX Security '10, USENIX Association, URL: https://www.usenix.org/legacy/event/sec10/tech/full_papers/Felmetsger.pdf.
- Gitea Project, 2026a. Release of gitea 1.25.4. URL: <https://blog.gitea.com/release-of-1.25.4/>, Official release notes; (Accessed 03 August 2026).
- Gitea Project, 2026b. Release of gitea 1.25.5. URL: <https://blog.gitea.com/release-of-1.25.5/>, Official release notes; (Accessed 03 August 2026).
- GORM Project, 2026. GORM documentation. URL: <https://gorm.io/docs/>, Official documentation; (Accessed 03 August 2026).
- Güler, E., Schumilo, S., Schloegel, M., Bars, N., Göz, P., Xu, X., Kaygusuz, C., Holz, T., 2024. Atropos: Effective fuzzing of web applications for server-side vulnerabilities. In: Proceedings of the 33rd USENIX Security Symposium. In: USENIX Security '24, USENIX Association, Philadelphia, PA, USA, pp. 1523–1538, URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/guler>.
- Guo, X., Kaway, A., Liu, E., Lie, D., 2025. EvoCrawl: Exploring web application code and state using evolutionary search. In: Proceedings of the 32nd Network and Distributed System Security Symposium. NDSS '25, The Internet Society, <http://dx.doi.org/10.14722/ndss.2025.230366>.
- Hibernate Team, 2026. Hibernate ORM user guide. URL: https://docs.hibernate.org/orm/current/userguide/html_single/Hibernate_User_Guide.html, Official documentation; (Accessed 03 August 2026).
- Hu, V.C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., Scarfone, K., 2014. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. Technical Report Special Publication 800-162, National Institute of Standards and Technology, <http://dx.doi.org/10.6028/NIST.SP.800-162>.
- Huang, Y., Shi, C., Lu, J., Li, H., Meng, H., Li, L., 2024. Detecting broken object-level authorization vulnerabilities in database-backed applications. In: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security. CCS '24, Association for Computing Machinery, New York, NY, USA, pp. 2934–2948. <http://dx.doi.org/10.1145/3658644.3690227>.
- JSqParser Project, 2026. JSqParser: Java SQL parser. URL: <https://jsqparser.github.io/JSqParser/>, (Accessed 29 May 2026), Software documentation.
- Kaur, B., 2026. Broken object level authorization in the wild: An empirical taxonomy from 100+ bug bounty disclosures. <http://dx.doi.org/10.48550/arXiv.2605.25865>, arXiv:2605.25865.
- Laravel LLC, 2026. Eloquent: Getting started. URL: <https://laravel.com/docs/12.x/eloquent>, Official Laravel documentation; (Accessed 03 August 2026).
- Liu, F., Shi, Y., Zhang, Y., Yang, G., Li, E., Yang, M., 2025a. MOCGuard: Automatically detecting missing-owner-check vulnerabilities in Java web applications. In: Proceedings of the 2025 IEEE Symposium on Security and Privacy. In: IEEE S&P '25, IEEE Computer Society, pp. 903–919. <http://dx.doi.org/10.1109/SP61157.2025.00010>.
- Liu, F., Zhang, Y., Li, E., Meng, W., Shi, Y., Wang, Q., Wang, C., Lin, Z., Yang, M., 2025b. BACScan: Automatic black-box detection of broken-access-control vulnerabilities in web applications. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. CCS '25, Association for Computing Machinery, New York, NY, USA, pp. 1320–1333. <http://dx.doi.org/10.1145/3719027.3744825>.
- Mehta, A., Elnikety, E., Harvey, K., Garg, D., Druschel, P., 2017. Qapla: Policy compliance for database-backed systems. In: 26th USENIX Security Symposium. In: USENIX Security '17, USENIX Association, Vancouver, BC, Canada, pp. 1463–1479, URL: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/mehta>.
- Microsoft, 2025. Row-Level Security. URL: <https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security?view=sql-server-ver17>, (Accessed 30 May 2026), Microsoft SQL Server Documentation.
- MITRE Corporation, 2026a. CWE-284: Improper Access Control. URL: <https://cwe.mitre.org/data/definitions/284.html>, (Accessed 30 May 2026), Common Weakness Enumeration.
- MITRE Corporation, 2026b. CWE-639: Authorization Bypass Through User-Controlled Key. URL: <https://cwe.mitre.org/data/definitions/639.html>, (Accessed 30 May 2026), Common Weakness Enumeration.
- Monshizadeh, M., Naldurg, P., Venkatakrishnan, V.N., 2014. MACE: Detecting privilege escalation vulnerabilities in web applications. In: Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security. CCS '14, Association for Computing Machinery, <http://dx.doi.org/10.1145/2660267.2660337>.
- National Vulnerability Database, 2023. CVE-2023-33779 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2023-33779>, (Accessed 20 June 2026).
- National Vulnerability Database, 2025a. CVE-2025-28400 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2025-28400>, (Accessed 20 June 2026).
- National Vulnerability Database, 2025b. CVE-2025-28402 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2025-28402>, (Accessed 20 June 2026).
- National Vulnerability Database, 2025c. CVE-2025-28406 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2025-28406>, (Accessed 20 June 2026).
- National Vulnerability Database, 2025d. CVE-2025-28407 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2025-28407>, (Accessed 20 June 2026).
- National Vulnerability Database, 2025e. CVE-2025-28411 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2025-28411>, (Accessed 20 June 2026).
- National Vulnerability Database, 2025f. CVE-2025-28412 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2025-28412>, (Accessed 20 June 2026).
- National Vulnerability Database, 2025g. CVE-2025-28413 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2025-28413>, (Accessed 20 June 2026).
- National Vulnerability Database, 2026a. CVE-2026-20904 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2026-20904>, (Accessed 03 August 2026).
- National Vulnerability Database, 2026b. CVE-2026-27835 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2026-27835>, (Accessed 03 August 2026).
- National Vulnerability Database, 2026c. CVE-2026-27839 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2026-27839>, (Accessed 03 August 2026).
- National Vulnerability Database, 2026d. CVE-2026-33053 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2026-33053>, (Accessed 07 August 2026).
- National Vulnerability Database, 2026e. CVE-2026-34046 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2026-34046>, (Accessed 07 August 2026).
- National Vulnerability Database, 2026f. CVE-2026-38530 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2026-38530>, (Accessed 07 August 2026).
- National Vulnerability Database, 2026g. CVE-2026-38532 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2026-38532>, (Accessed 07 August 2026).
- National Vulnerability Database, 2026h. CVE-2026-52839 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2026-52839>, (Accessed 07 August 2026).
- National Vulnerability Database, 2026i. CVE-2026-55255 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2026-55255>, (Accessed 07 August 2026).
- National Vulnerability Database, 2026j. CVE-2026-61460 Detail. URL: <https://nvd.nist.gov/vuln/detail/CVE-2026-61460>, (Accessed 07 August 2026).
- OWASP Foundation, 2021. A01:2021 – Broken Access Control. URL: https://owasp.org/Top10/A01_2021-Broken_Access_Control/, (Accessed 29 May 2026), OWASP Top 10.
- OWASP Foundation, 2023a. API1:2023 – Broken Object Level Authorization. URL: <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>, (Accessed 29 May 2026), OWASP API Security Top 10.
- OWASP Foundation, 2023b. API3:2023 – Broken Object Property Level Authorization. URL: <https://owasp.org/API-Security/editions/2023/en/0xa3-broken-object-property-level-authorization/>, (Accessed 30 May 2026), OWASP API Security Top 10.
- OWASP Foundation, 2023c. API5:2023 – Broken Function Level Authorization. URL: <https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/>, (Accessed 29 May 2026), OWASP API Security Top 10.
- OWASP Foundation, 2025. A01:2025 – Broken Access Control. URL: https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/, (Accessed 29 May 2026), OWASP Top 10.
- Pappachan, P., Yus, R., Mehrotra, S., Freytag, J.-C., 2020. Sieve: A middleware approach to scalable access control for database management systems. Proc. the VLDB Endow. 13 (11), 2424–2437. <http://dx.doi.org/10.14778/3407790.3407835>.
- Pellegrino, G., Balzarotti, D., 2014. Toward black-box detection of logic flaws in web applications. In: Proceedings of the Network and Distributed System Security Symposium. NDSS '14, URL: https://www.ndss-symposium.org/wp-content/uploads/2017/09/04_2_1.pdf.
- PortSwigger Web Security Academy, 2026a. Access control vulnerabilities and privilege escalation. URL: <https://portswigger.net/web-security/access-control>, (Accessed 29 May 2026), Web Security Academy.
- PortSwigger Web Security Academy, 2026b. Insecure direct object references. URL: <https://portswigger.net/web-security/access-control/idor>, (Accessed 29 May 2026), Web Security Academy.
- PostgreSQL Global Development Group, 2026. PostgreSQL documentation: Row security policies. URL: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>, (Accessed 30 May 2026), PostgreSQL Documentation.
- Rizvi, S., Mendelzon, A., Sudarshan, S., Roy, P., 2004. Extending query rewriting techniques for fine-grained access control. In: Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data. SIGMOD '04, Association for Computing Machinery, pp. 551–562. <http://dx.doi.org/10.1145/1007568.1007631>.
- RuoYi Project, 2026. RuoYi: A Spring Boot-Based Permission Management System. URL: <https://github.com/yangzongzhuan/RuoYi>, (Accessed 20 June 2026), GitHub repository.
- Sandhu, R.S., Coyne, E.J., Feinstein, H.L., Youman, C.E., 1996. Role-based access control models. Computer 29 (2), 38–47. <http://dx.doi.org/10.1109/2.485845>.
- Schlaubitz, M., Veyisoglu, O., Rennhard, M., 2025. A2CT: Automated detection of function and object-level access control vulnerabilities in web applications. In: Proceedings of the 11th International Conference on Information Systems Security and Privacy. ICISPP '25, SciTePress, ISBN: 978-989-758-735-1, pp. 425–436. <http://dx.doi.org/10.5220/0013092700003899>.
- Shay, R., Blumenthal, U., Gadepally, V., Hamlin, A., Darby, J., Cunningham, R.K., 2018. Don't even ask: Database access control through query control. SIGMOD Rec. 47 (3), 20–25. <http://dx.doi.org/10.1145/3316416.3316420>.
- SQLAlchemy Authors, 2026. SQLAlchemy ORM documentation. URL: <https://docs.sqlalchemy.org/en/20/orm/>, Official documentation; (Accessed 03 August 2026).
- Sun, F., Xu, L., Su, Z., 2011. Static detection of access control vulnerabilities in web applications. In: Proceedings of the 20th USENIX Security Symposium. In: USENIX Security '11, USENIX Association, URL: <https://www.usenix.org/conference/usenix-security-11/static-detection-access-control-vulnerabilities-web-applications>.

- Wang, M., Götz, P., Schilling, J., Hassler, K., Guo, L., Holz, T., Abbasi, A., 2026. Anota: Identifying business logic vulnerabilities via annotation-based sanitization. In: Proceedings of the 33rd Network and Distributed System Security Symposium. NDSS '26, The Internet Society, URL: <https://www.ndss-symposium.org/ndss-paper/anota-identifying-business-logic-vulnerabilities-via-annotation-based-sanitization/>.
- Wang, C., Meng, W., Luo, C., Li, P., 2025. Predator: Directed web application fuzzing for efficient vulnerability validation. In: Proceedings of the 2025 IEEE Symposium on Security and Privacy. In: IEEE S&P '25, IEEE Computer Society, pp. 886–902. <http://dx.doi.org/10.1109/SP61157.2025.00066>.
- World Wide Web Consortium, 2021. Trace Context. URL: <https://www.w3.org/TR/trace-context/>, (Accessed 30 May 2026), W3C Recommendation.
- Wu, A., Feng, Z., Feng, R., Xing, Z., Liu, Y., 2025. Rethinking broken object level authorization attacks under zero trust principle. <http://dx.doi.org/10.48550/arXiv.2507.02309>, arXiv:2507.02309.
- XORM Authors, 2026a. XORM conditions. URL: <https://xorm.io/docs/chapter-05/1.conditions/>, Official XORM documentation; (Accessed 03 August 2026).
- XORM Authors, 2026b. XORM database operations. URL: <https://xorm.io/docs/chapter-06/readme/>, Official XORM documentation; (Accessed 03 August 2026).
- Xu, K., Zhang, B., Li, J., He, H., Ren, R., Ren, J., 2025. DRacv: Detecting and auto-repairing vulnerabilities in role-based access control in web application. J. Netw. Comput. Appl. 240, 104191. <http://dx.doi.org/10.1016/j.jnca.2025.104191>.
- XXL-JOB Project, 2026. XXL-job: A distributed task scheduling platform. URL: <https://github.com/xuxueli/xxl-job>, (Accessed 20 June 2026), GitHub repository.
- Zhang, W., Bali, D., Kerney, J., Panda, A., Shenker, S., 2023. Access control for database applications: Beyond policy enforcement. In: Proceedings of the Workshop on Hot Topics in Operating Systems. HotOS '23, Association for Computing Machinery, <http://dx.doi.org/10.1145/3593856.3595905>.
- Zhang, W., Bali, D., Kerney, J., Panda, A., Shenker, S., 2024. Extracting database access-control policies from web applications. URL: <https://arxiv.org/abs/2411.11380>, arXiv preprint arXiv:2411.11380.
- Zhang, W., Bali, D.A., Panda, A., Shenker, S., 2022. Blockaid: Data access policy enforcement for web applications. In: Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation. OSDI '22, USENIX Association, URL: <https://www.usenix.org/conference/osdi22/presentation/zhang>.
- Zhang, Q., Liu, F., Lin, Z., Zhang, Y., 2025. Be aware of what you let pass: Demystifying URL-based authentication bypass vulnerability in Java web applications. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. CCS '25, Association for Computing Machinery, New York, NY, USA, pp. 1874–1888. <http://dx.doi.org/10.1145/3719027.3765199>.
- Zuo, C., Zhao, Q., Lin, Z., 2017. AuthScope: Towards automatic discovery of vulnerable authorizations in online services. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. CCS '17, Association for Computing Machinery, pp. 799–813. <http://dx.doi.org/10.1145/3133956.3134089>.